

TRƯỜNG ĐẠI HỌC BÁCH KHOA - ĐHQG TP.HCM
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



Dữ liệu lớn – CO3137

Báo cáo Bài tập lớn – Nhóm BDCon

**Trino: Fast Distributed SQL Query Engine
for Big Data Analytics**

Giảng viên: TS. Nguyễn Quang Hùng

Sinh viên:	Ngô Ngọc Tuấn Anh	MSSV: 2210078	L01
	Lê Đình Đức	MSSV: 2310774	L01
	Nguyễn Văn Công Thành	MSSV: 2313133	L01
	Nguyễn Phúc Nhân	MSSV: 2312438	L01

TP. HỒ CHÍ MINH, THÁNG 7/2025



Danh sách thành viên và phân công nhiệm vụ

STT	Họ và tên	MSSV	Nhiệm vụ chính
1	Ngô Ngọc Tuấn Anh	2210078	Thiết kế kiến trúc Medallion và xây dựng các luồng ETL.
2	Lê Đình Đức	2310774	Thiết kế kiến trúc hạ tầng cho Data Platform
3	Nguyễn Văn Công Thành	2313133	Xây dựng pipeline benchmark TPC-DS, phát triển bộ thu thập Metrics.
4	Nguyễn Phúc Nhân	2312438	Xây dựng pipeline benchmark TPC-DS, tính toán và trình bày kết quả

Contents

Danh sách thành viên và phân công nhiệm vụ	1
1 Giới Thiệu	4
1.1 Bối cảnh và Động lực	4
1.2 Mục tiêu Báo cáo	4
1.3 Phạm vi Dự án	4
2 Cơ sở Lý thuyết	6
2.1 Big Data và Data Lakehouse	6
2.1.1 Khái niệm Big Data	6
2.1.2 Khái niệm Data Lakehouse	6
2.1.3 Kiến trúc Medallion	7
2.2 Apache Trino	7
2.2.1 Tổng quan Trino	7
2.2.2 Kiến trúc MPP của Trino	8
2.2.3 Query Execution Pipeline	9
2.2.4 Tối ưu truy vấn trong Trino	10
2.2.5 Connector Architecture	11
2.2.6 So sánh Trino với các Query Engine khác	12
3 Dữ liệu và Kiến trúc Hệ thống	18
3.1 Chuẩn bị Dữ liệu	18
3.1.1 theLook Ecommerce	18
3.1.2 Kiến trúc Medallion và Data Lineage	19
3.1.3 TPC-DS	19
3.2 Xây dựng Hệ thống	20
3.2.1 Tổng quan Nền tảng	20
3.2.2 Cơ sở Hạ tầng (Infrastructure Layer)	20
3.2.3 Storage và Catalog Layer	23
3.2.4 Streaming và Processing Layer	24
3.2.5 Triển khai Trino trên Kubernetes	24
3.2.6 BI và Truy cập Hệ thống	25
3.2.7 Tóm tắt Cấu hình Triển khai	26
4 Hiện thực	27
4.1 Truy vấn Đa nguồn	27
4.2 Truy vấn Phân tán	29
4.3 Tích hợp với Superset trong trực quan hóa dữ liệu	30
5 Benchmark	32
5.1 Mục tiêu Benchmark	32
5.2 Dữ liệu và Catalog	32
5.2.1 TPC-DS	32
5.2.2 MinIO Distributed Storage	32
5.3 Cấu hình Engine	33
5.3.1 Trino Benchmark	33
5.3.2 Spark Benchmark	33
5.4 Quy trình Chạy Benchmark	33
5.5 Metrics Thu thập	34
5.6 Kết quả Benchmark	34
5.6.1 Hiệu năng tổng thể – Geometric Mean Latency	35
5.6.2 Phân tích Latency theo từng Query	36
5.6.3 Speedup Ratio – Lợi thế Tương đối của Trino	37
5.6.4 Peak Memory Usage – Hiệu quả Sử dụng Bộ nhớ	37
5.6.5 Nhận xét và Kết luận Benchmark	38
5.6.6 Hạn chế của Benchmark	38
6 Tổng kết	39



6.1	Những kết quả đạt được	39
6.2	Hạn chế còn tồn tại	39
6.3	Hướng phát triển tương lai	39

1 Giới Thiệu

1.1 Bối cảnh và Động lực

Trong kỷ nguyên dữ liệu lớn (Big Data), các tổ chức hiện đại phải đối mặt với thách thức ngày càng tăng trong việc truy vấn và phân tích khối lượng dữ liệu khổng lồ lưu trữ trên các hệ thống phân tán. Các hệ thống cơ sở dữ liệu truyền thống không còn đáp ứng được yêu cầu về tốc độ và khả năng mở rộng khi phải xử lý petabytes dữ liệu.

Apache Trino (tiền thân là PrestoSQL) là một distributed SQL query engine mã nguồn mở, được thiết kế để chạy các truy vấn SQL tốc độ cao trên các tập dữ liệu khổng lồ. Trino tách biệt tầng tính toán (compute) ra khỏi tầng lưu trữ (storage), cho phép truy vấn dữ liệu từ nhiều nguồn khác nhau mà không cần phải di chuyển hay sao chép dữ liệu.



Figure 1: Apache Trino trong vai trò distributed SQL query engine cho dữ liệu phân tán

1.2 Mục tiêu Báo cáo

Báo cáo này trình bày:

- (1) Kiến trúc và nguyên lý hoạt động của Trino trong môi trường phân tán.
- (2) Triển khai Trino trong một nền tảng Data Platform thực tế trên Kubernetes (GKE).
- (3) So sánh Trino với các công cụ truy vấn tương đương trên dữ liệu lớn.
- (4) Benchmark và so sánh hiệu năng giữa Trino và Apache Spark trên bộ dữ liệu TPC-DS.
- (5) Đánh giá điểm mạnh, hạn chế và trường hợp sử dụng phù hợp của Trino.

1.3 Phạm vi Dự án

Nhóm nghiên cứu và triển khai một hệ thống **Modern Data Platform (Lakehouse)** hoàn chỉnh trên nền tảng Google Kubernetes Engine (GKE). Kiến trúc này là sự kết hợp của các thành phần cốt lõi nhằm đảm bảo tính linh hoạt và khả năng mở rộng:

- **Lưu trữ:** MinIO (Object Storage) và Apache Iceberg (Table Format).
- **Xử lý dữ liệu:** Apache Spark cho xử lý lô (batch) và Apache Kafka cho xử lý dòng (streaming).
- **Quản lý & Truy vấn:** Apache Hive (Catalog), **Trino** (Query Engine).
- **Trực quan hóa:** Apache Superset.

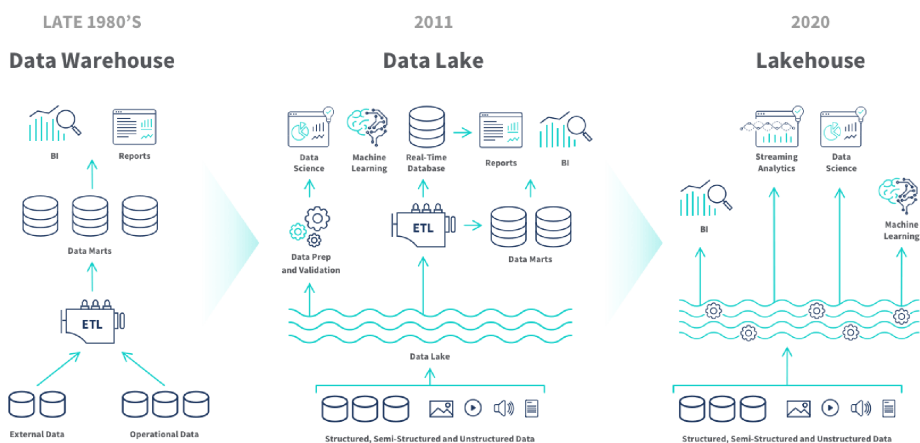


Figure 2: Lakehouse là kiến trúc nền tảng cho việc lưu trữ, xử lý và truy vấn dữ liệu trong đồ án

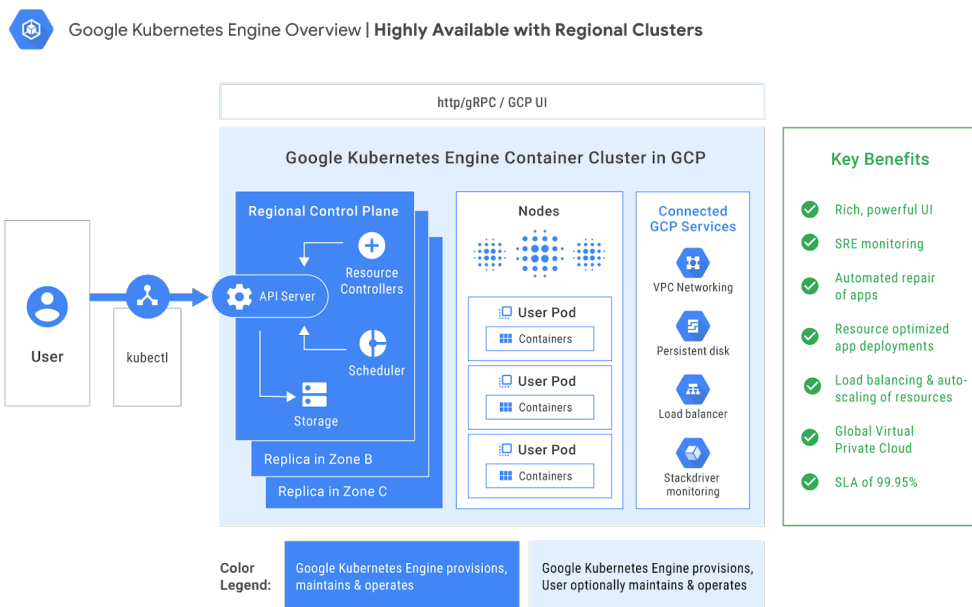


Figure 3: Google Kubernetes Engine - Công cụ điều phối và quản trị các node tài nguyên trên Google Cloud Platform

2 Cơ sở Lý thuyết

2.1 Big Data và Data Lakehouse

2.1.1 Khái niệm Big Data

Big Data là tập hợp dữ liệu có quy mô, tốc độ sinh ra, mức độ đa dạng và độ phức tạp vượt quá khả năng xử lý hiệu quả của các hệ thống quản trị dữ liệu truyền thống. Trong bối cảnh hiện đại, Big Data không chỉ có nghĩa là “dữ liệu rất lớn”, mà còn là năng lực thu thập, lưu trữ, quản trị và khai thác dữ liệu phân tán để tạo ra giá trị phân tích, tự động hóa và ra quyết định.

Các đặc trưng phổ biến của Big Data thường được mô tả bằng mô hình **5V**:

- **Volume**: Dữ liệu có khối lượng lớn, thường đạt mức terabytes, petabytes hoặc hơn.
- **Velocity**: Dữ liệu được sinh ra và cần xử lý với tốc độ cao, bao gồm batch, micro-batch và real-time streaming.
- **Variety**: Dữ liệu tồn tại ở nhiều dạng như structured, semi-structured và unstructured data.
- **Veracity**: Dữ liệu cần được kiểm chứng về độ tin cậy, nguồn gốc, chất lượng và tính nhất quán.
- **Value**: Mục tiêu cuối cùng là chuyển dữ liệu thành insight, mô hình dự đoán, sản phẩm dữ liệu hoặc quyết định kinh doanh.

Đến năm 2026, khái niệm Big Data dịch chuyển từ bài toán lưu trữ và xử lý khối lượng lớn sang bài toán xây dựng **nền tảng dữ liệu đáng tin cậy cho AI và phân tích thời gian gần thực**. Một hệ thống Big Data hiện đại thường có các xu hướng sau:

- (1) **Lakehouse-first**: Dữ liệu được lưu trên object storage với table format như Apache Iceberg, Delta Lake hoặc Apache Hudi để kết hợp tính linh hoạt của Data Lake và tính quản trị của Data Warehouse.
- (2) **Real-time và near real-time analytics**: Dữ liệu từ Kafka, CDC, IoT hoặc application events được phân tích nhanh hơn để phục vụ dashboard, alerting và operational analytics.
- (3) **AI-ready data**: Dữ liệu cần có metadata, lineage, semantic layer và quality checks để dùng cho machine learning, GenAI và agentic AI một cách an toàn.
- (4) **Governance by design**: Khi dữ liệu do AI tạo ra ngày càng nhiều, hệ thống phải kiểm soát nguồn gốc, quyền truy cập, chất lượng và chính sách sử dụng dữ liệu ngay trong pipeline.
- (5) **Data products và data contracts**: Dữ liệu được đóng gói như sản phẩm có schema, SLA, owner và contract rõ ràng thay vì chỉ là file hoặc bảng rời rạc.

Vì vậy, trong đồ án này, Big Data được hiểu theo nghĩa rộng hơn: đó là một kiến trúc dữ liệu phân tán có khả năng lưu trữ dữ liệu lớn trên MinIO/Iceberg, xử lý batch bằng Spark, streaming qua Kafka, và truy vấn tương tác đa nguồn bằng Trino.

2.1.2 Khái niệm Data Lakehouse

Data Lakehouse là kiến trúc dữ liệu lai kết hợp tính linh hoạt và chi phí thấp của Data Lake với các tính năng quản lý dữ liệu có cấu trúc của Data Warehouse. Kiến trúc này cho phép:

- Lưu trữ dữ liệu thô (raw data) trực tiếp trên object storage (S3/MinIO/GCS/...).
- Hỗ trợ giao dịch ACID thông qua table format (Iceberg/Delta Lake/Hudi).
- Truy vấn SQL tốc độ cao thông qua query engine (Trino, Spark, Flink, Impala).
- Xử lý batch và streaming trên cùng một nền tảng.
- Hỗ trợ nhiều mục đích đầu cuối như BI, ML, AI.

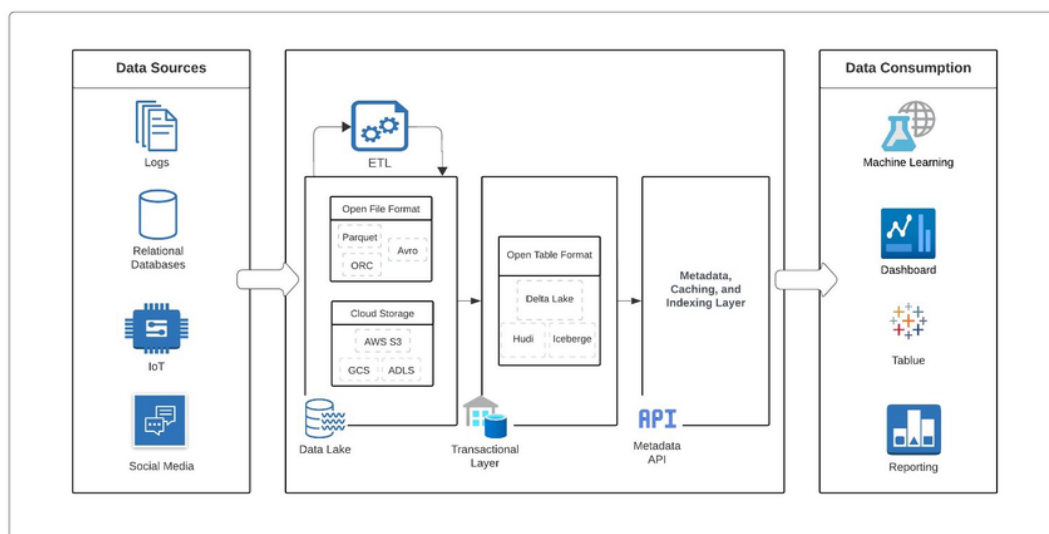


Figure 4: Tổng quan các thành phần, các lớp có trong Data Lakehouse

2.1.3 Kiến trúc Medallion

Kiến trúc Medallion tổ chức dữ liệu thành ba tầng logic với chất lượng và độ tin cậy tăng dần:

- **Tầng Bronze:** Lưu trữ dữ liệu nguyên bản từ các nguồn cung cấp (PostgreSQL, Kafka, API...). Dữ liệu tại đây được giữ nguyên trạng, mang tính bất biến (immutable) và đóng vai trò là kho lịch sử phục vụ việc tái xử lý (reprocessing).
- **Tầng Silver:** Dữ liệu từ tầng Bronze được làm sạch, loại bỏ trùng lặp (deduplication) và chuẩn hóa cấu trúc. Đây là "nguồn sự thật duy nhất" (Single Source of Truth), cung cấp dữ liệu tin cậy cho các tác vụ phân tích và học máy.
- **Tầng Gold:** Dữ liệu được tổng hợp, tính toán các chỉ số (metrics) và tổ chức theo các mô hình phân tích. Tầng này được tối ưu hóa về cấu trúc và hiệu năng để phục vụ trực tiếp cho báo cáo BI và các quyết định nghiệp vụ.

2.2 Apache Trino

2.2.1 Tổng quan Trino

Trino (trước đây là PrestoSQL) là một công cụ truy vấn SQL phân tán (distributed SQL query engine) mã nguồn mở, được thiết kế theo kiến trúc *MPP* (*Massively Parallel Processing*) nhằm thực hiện các truy vấn phân tích hiệu năng cao trên quy mô dữ liệu cực lớn.

- **Truy vấn tức thời (Ad-hoc queries):** Trino được tối ưu để thực thi các truy vấn có độ trễ cực thấp (từ mức dưới giây đến vài giây) nhờ vào cơ chế xử lý hoàn toàn trên bộ nhớ (in-memory processing) và mô hình thực thi đường ống (pipelined execution). Điều này giúp các nhà phân tích dữ liệu có thể tương tác trực tiếp với dữ liệu lớn một cách nhanh chóng mà không phải chờ đợi các tiến trình xử lý lô (batch processing) kéo dài.
- **Truy vấn liên kết (Federated queries):** Một trong những sức mạnh cốt lõi của Trino là kiến trúc dựa trên Connector, cho phép thực hiện các truy vấn tham chiếu đồng thời đến nhiều nguồn dữ liệu khác nhau trong cùng một câu lệnh SQL. Trino có thể kết hợp (join) dữ liệu từ các hệ thống không đồng nhất như: các Data Lake (S3, MinIO, HDFS thông qua Hive/Iceberg), các cơ sở dữ liệu quan hệ (PostgreSQL, MySQL), cơ sở dữ liệu NoSQL (MongoDB, Cassandra) và thậm chí là các nền tảng streaming (Kafka).
- **Xử lý phân tích trực tuyến (OLAP workloads):** Trino hỗ trợ các tác vụ phân tích dữ liệu quy mô lớn với độ trễ thấp trên hàng petabyte dữ liệu. Nhờ vào khả năng mở rộng ngang (horizontal scaling) của các Worker node và cơ chế tối ưu hóa truy vấn dựa trên chi phí (Cost-Based Optimizer - CBO), Trino phân phối khối lượng công việc tính toán một cách hiệu quả, giúp giảm thiểu tối đa thời gian phản hồi cho các báo cáo BI và bảng điều khiển (dashboards) phức tạp.

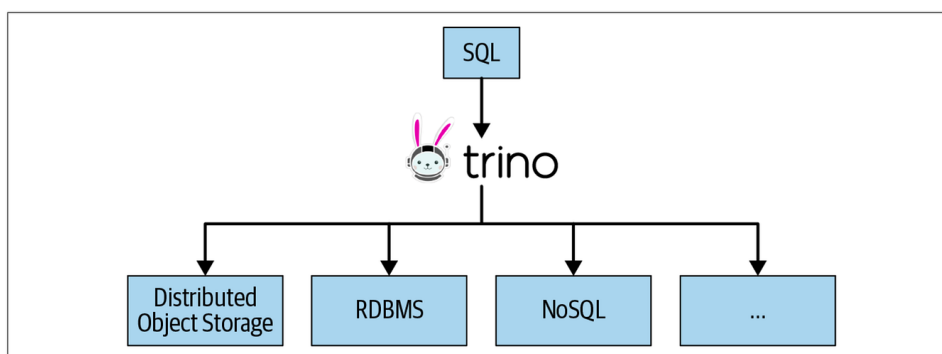


Figure 5: Trino đóng vai trò lớp SQL tương tác và liên hợp phía trên nhiều nguồn dữ liệu

Một **distributed query engine** thực thi SQL bằng cách chia truy vấn thành nhiều phần việc nhỏ và chạy song song trên nhiều node. Coordinator lập kế hoạch, chia dữ liệu thành các splits, phân phối tasks cho workers và tổng hợp kết quả trả về. Nhờ khả năng mở rộng theo chiều ngang, Trino có thể tăng năng lực truy vấn trên dữ liệu lớn bằng cách bổ sung thêm worker node thay vì phụ thuộc vào một máy chủ đơn lẻ.

Trong phân tích dữ liệu lớn, Trino đóng vai trò **query serving layer**: Spark hoặc pipeline batch/streaming xử lý ingest và transformation; Iceberg/object storage lưu dữ liệu lâu dài; Trino cung cấp lớp SQL tương tác để truy vấn nhanh trên Lakehouse, PostgreSQL, Kafka và các nguồn khác. Trino không lưu dữ liệu nội bộ, phù hợp với mô hình **separation of compute and storage**, giúp hạn chế sao chép dữ liệu sang nhiều data mart riêng.

Mô hình namespace của Trino gồm ba cấp chính:

- **Catalog**: Đại diện cho một connector cụ thể, ví dụ iceberg, hive, postgresql, kafka.
- **Schema**: Nhóm các bảng trong một catalog, tương tự database/schema trong các hệ SQL.
- **Table/View**: Đối tượng dữ liệu được truy vấn bằng SQL.

Với mô hình này, người dùng có thể tham chiếu dữ liệu bằng tên đầy đủ dạng `catalog.schema.table`. Một câu SQL trong Trino có thể join dữ liệu giữa nhiều catalog, ví dụ join bảng Iceberg trên MinIO với bảng metadata trong PostgreSQL mà không cần ETL dữ liệu về cùng một nơi trước.

2.2.2 Kiến trúc MPP của Trino

Trino sử dụng kiến trúc **Massively Parallel Processing (MPP)**:

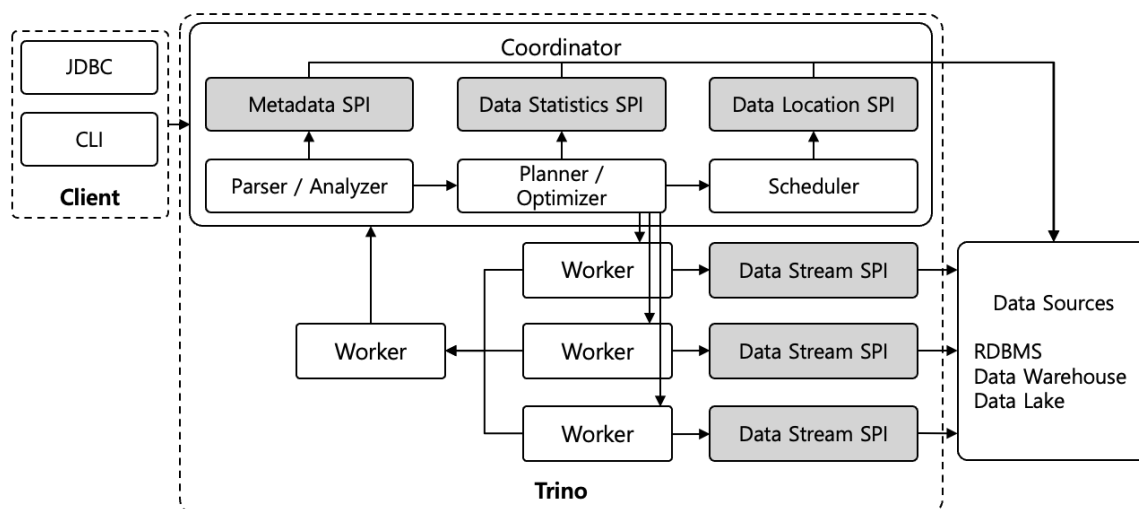


Figure 6: Kiến trúc MPP của Trino: coordinator lập kế hoạch, workers thực thi song song và đọc dữ liệu qua connectors

2.2.2.1 Coordinator:

- Tiếp nhận câu truy vấn SQL từ client.
- Phân tích và tối ưu hóa query plan (Cost-Based Optimizer).
- Phân chia query thành các *stages* và *tasks*, phân phối cho Workers.
- Thu thập kết quả và trả về client.

2.2.2.2 Workers:

- Thực thi các *tasks* được giao bởi Coordinator.
- Đọc dữ liệu trực tiếp từ data source thông qua Connectors.
- Trao đổi intermediate data giữa các Workers thông qua *Exchange*.
- Sử dụng bộ nhớ RAM để pipeline dữ liệu, tránh ghi đĩa khi có thể.

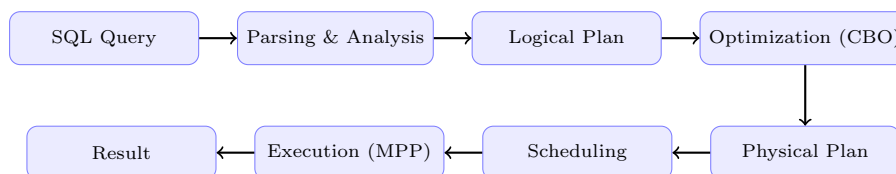
Trong quá trình thực thi, Trino chia truy vấn thành nhiều đơn vị nhỏ:

- **Stage:** Một phần của physical plan, thường tương ứng với một pha xử lý như scan, join, aggregation hoặc final output.
- **Task:** Đơn vị thực thi của một stage trên một worker cụ thể.
- **Split:** Phần dữ liệu nhỏ do connector cung cấp để worker đọc song song, ví dụ một vùng file Parquet/Iceberg hoặc một partition dữ liệu.
- **Driver/Operator:** Pipeline xử lý bên trong task, gồm các operator như table scan, filter, project, join, aggregate.

Điểm quan trọng của Trino là xử lý dữ liệu theo kiểu **pipelined execution**. Khi có thể, dữ liệu được truyền qua các operator và exchange trong bộ nhớ thay vì ghi toàn bộ kết quả trung gian xuống đĩa. Cách này giúp giảm độ trễ cho truy vấn tương tác, nhưng cũng khiến việc quản lý memory, join distribution và kích thước dữ liệu trung gian trở thành yếu tố rất quan trọng.

2.2.3 Query Execution Pipeline

Một câu truy vấn Trino đi qua các bước sau:



Các bước trong pipeline có thể diễn giải như sau:

- (1) **Parsing:** Trino phân tích cú pháp SQL và tạo abstract syntax tree.
- (2) **Analysis:** Kiểm tra catalog, schema, table, column, kiểu dữ liệu và quyền truy cập.
- (3) **Logical planning:** Chuyển câu SQL thành logical plan gồm scan, filter, project, join, aggregation.
- (4) **Optimization:** Áp dụng các rule-based và cost-based optimization để giảm chi phí thực thi.
- (5) **Physical planning:** Chuyển logical plan thành distributed plan gồm fragments/stages và cách phân phối dữ liệu.
- (6) **Scheduling:** Coordinator phân phối tasks cho workers dựa trên splits, tài nguyên và trạng thái cluster.
- (7) **Execution:** Workers đọc dữ liệu, xử lý operator pipeline, trao đổi dữ liệu qua exchange và trả kết quả về coordinator/client.

2.2.4 Tối ưu truy vấn trong Trino

Hiệu năng của Trino phụ thuộc đồng thời vào query optimizer, khả năng pushdown của connector, layout dữ liệu vật lý và cách viết SQL. Các nhóm tối ưu quan trọng gồm:

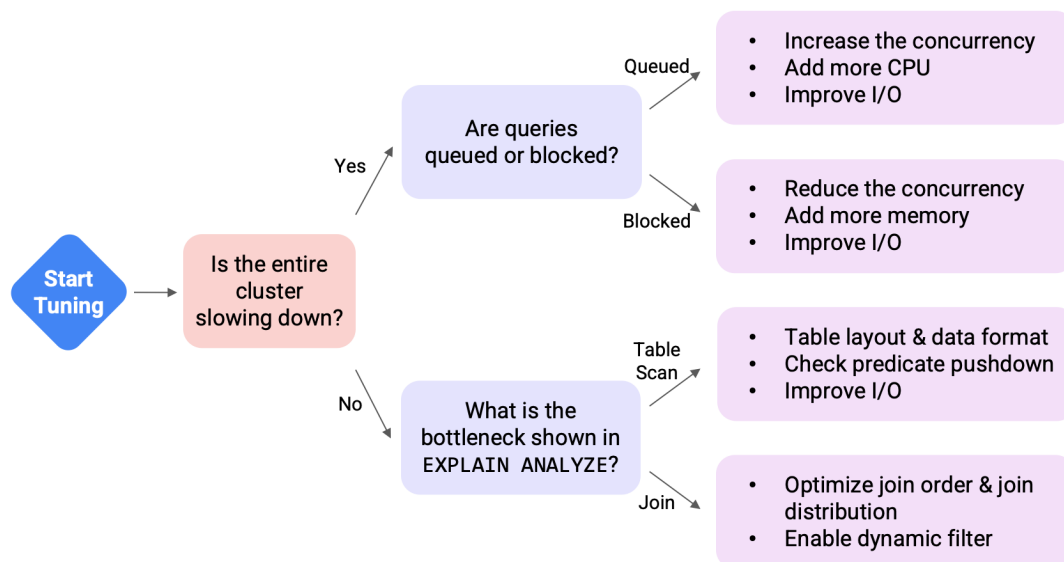


Figure 7: Quy trình ra quyết định và các chiến lược tối ưu hóa hiệu năng (Performance Tuning) cho Trino

2.2.4.1 Cost-Based Optimizer (CBO): Trino sử dụng thống kê bảng/cột để ước lượng số dòng, kích thước dữ liệu, chi phí CPU, memory và network của các phương án thực thi. CBO đặc biệt quan trọng với các truy vấn có nhiều join vì thứ tự join ảnh hưởng trực tiếp đến lượng dữ liệu trung gian. Khi có statistics tốt, Trino có thể tự động chọn join order rẻ hơn, tránh thực hiện join sinh dữ liệu lớn quá sớm.

- Với Iceberg/Hive, cần duy trì metadata và statistics chính xác để optimizer ước lượng tốt hơn.
- Với truy vấn nhiều bảng, nên kiểm tra plan để đảm bảo bảng nhỏ/lọc mạnh được xử lý sớm.
- Nếu statistics thiếu hoặc sai, optimizer có thể chọn join order hoặc join distribution chưa tối ưu.

2.2.4.2 Predicate, Projection và Aggregation Pushdown: Pushdown là cơ chế đẩy một phần xử lý xuống data source thông qua connector. Thay vì đọc toàn bộ dữ liệu về Trino rồi mới lọc/tính toán, connector có thể yêu cầu nguồn dữ liệu chỉ trả về phần cần thiết.

- **Predicate pushdown:** Đẩy điều kiện WHERE xuống nguồn để giảm số dòng đọc.
- **Projection pushdown:** Chỉ đọc các cột xuất hiện trong SELECT, JOIN, WHERE, GROUP BY.
- **Aggregation pushdown:** Một số connector có thể đẩy COUNT, SUM, GROUP BY xuống nguồn.
- **Join pushdown:** Một số connector có thể đẩy join xuống data source nếu hai bảng nằm cùng catalog và nguồn hỗ trợ xử lý join hiệu quả.

Trong Lakehouse, predicate/projection pushdown giúp giảm lượng file, row group và column chunk phải đọc từ Parquet/ORC. Với JDBC connector như PostgreSQL, pushdown giúp giảm network traffic giữa database và Trino.

2.2.4.3 Dynamic Filtering: Dynamic filtering tối ưu các truy vấn join giữa bảng fact lớn và bảng dimension đã được lọc. Trong khi thực thi, Trino thu thập tập giá trị từ build side của join và dùng tập giá trị đó để lọc sớm probe side. Ví dụ, nếu bảng date_dim chỉ còn vài ngày sau điều kiện WHERE, Trino có thể dùng các khóa ngày này để tránh scan toàn bộ store_sales.

Cơ chế này đặc biệt hữu ích với star schema, TPC-DS, dashboard BI và các truy vấn có dimension filter chọn lọc cao. Tuy nhiên, hiệu quả phụ thuộc vào connector, kiểu join, kích thước build side và khả năng áp dụng filter xuống table scan.

2.2.4.4 Join Distribution và Join Reordering: Trino thường dùng hash join và có hai kiểu phân phối chính:

- **Broadcast join:** Bảng nhỏ được gửi tới tất cả workers; phù hợp khi một phía join đủ nhỏ.
- **Partitioned join:** Hai phía join được shuffle theo join key; phù hợp khi cả hai phía đều lớn.

Broadcast join có thể giảm shuffle cho bảng lớn nhưng tốn memory vì mỗi worker giữ một bản sao bảng nhỏ. Partitioned join phân tán dữ liệu tốt hơn cho join lớn, nhưng network shuffle cao hơn. Vì vậy, statistics và cấu hình memory ảnh hưởng mạnh đến lựa chọn join distribution.

2.2.4.5 Tối ưu dữ liệu Lakehouse: Với dữ liệu Iceberg/Parquet trên MinIO, tối ưu query không chỉ nằm ở Trino mà còn nằm ở layout dữ liệu:

- Chọn partition phù hợp với filter thường gặp, tránh partition quá nhỏ hoặc quá nhiều.
- Dùng Parquet/ORC để tận dụng columnar scan, predicate pushdown và compression.
- Tránh small files vì quá nhiều file làm tăng overhead listing, planning và scheduling splits.
- Chạy compaction/optimize để gom file nhỏ và cải thiện kích thước split.
- Cập nhật table statistics/metadata để CBO có thông tin chính xác.

2.2.4.6 Tối ưu cách viết SQL: Một số nguyên tắc viết query giúp Trino tối ưu tốt hơn:

- Chỉ chọn cột cần thiết thay vì **SELECT ***.
- Đặt điều kiện lọc càng sớm càng tốt và dùng kiểu dữ liệu nhất quán để tránh cast không cần thiết.
- Tránh biểu thức phức tạp trực tiếp trên cột lọc nếu biểu thức đó làm giảm khả năng pushdown.
- Với join lớn, đảm bảo join key có kiểu dữ liệu giống nhau và tránh join trên biểu thức khó tối ưu.
- Dùng **LIMIT** cho truy vấn khám phá dữ liệu, nhưng không kỳ vọng **LIMIT** luôn giảm chi phí nếu plan vẫn phải sort/aggregate toàn cục.
- Với truy vấn lặp lại cho BI, cân nhắc tạo bảng Gold/materialized result bằng pipeline Spark hoặc Iceberg để giảm tính toán lại.

2.2.4.7 Phân tích query plan bằng EXPLAIN: Trino cung cấp **EXPLAIN** và **EXPLAIN ANALYZE** để kiểm tra plan. **EXPLAIN** cho biết distributed plan, fragment, join distribution, table scan và các operator chính. **EXPLAIN ANALYZE** thực thi truy vấn và trả về thống kê thực tế như CPU time, scheduled time, blocked time, số dòng input/output và mức độ lệch dữ liệu giữa tasks.

Khi phân tích query chậm, cần chú ý:

- Table scan có đọc quá nhiều cột/dòng không.
- Predicate, projection hoặc aggregation có được pushdown không.
- Join có tạo dữ liệu trung gian quá lớn không.
- Có skew giữa các tasks/workers không.
- Thời gian bị nghẽn ở CPU, memory, network exchange hay đọc object storage.

2.2.5 Connector Architecture

Trino giao tiếp với các data source thông qua **Connector API**. Mỗi connector cung cấp:

- **Metadata API:** Schema, bảng, cột, thống kê.
- **Data Location API:** Phân chia dữ liệu thành các splits để đọc song song.
- **Data Source API:** Đọc dữ liệu thực tế.

Trong dự án, các connector được sử dụng:

- **Iceberg connector:** Truy vấn bằng Iceberg trên MinIO thông qua Gravitino REST Catalog.
- **Hive connector:** Tương thích ngược với Hive Metastore.
- **Kafka connector:** Truy vấn trực tiếp topic Kafka dưới dạng bảng.
- **TPC-DS connector:** Tạo dữ liệu benchmark chuẩn.

2.2.6 So sánh Trino với các Query Engine khác

Trong hệ sinh thái dữ liệu lớn đến năm 2026, ranh giới giữa query engine, processing engine và OLAP serving engine không còn hoàn toàn tách biệt. Spark SQL ngày càng tối ưu cho Lakehouse, Hive vẫn giữ vai trò metadata/warehouse quan trọng, Impala tiếp tục phục vụ low-latency SQL trong các cụm Hadoop/S3 đã chuẩn hóa, còn Doris mở rộng mạnh sang lakehouse acceleration thông qua external catalog và materialized view. Tuy nhiên, mỗi công cụ vẫn có trọng tâm thiết kế khác nhau. Vì vậy, phần này so sánh từng công cụ trực tiếp với Trino thay vì gom vào một bảng tổng hợp.

Trino trong bối cảnh mới được xem là **federated interactive query engine**: engine tập trung vào SQL tương tác, kiến trúc MPP, connector phong phú, đọc dữ liệu tại chỗ trên Iceberg/Hive/object storage/JDBC/Kafka và có thêm các cơ chế như metadata caching, file-system caching, dynamic filtering và fault-tolerant execution cho một số workload/connector. Điểm mạnh của Trino không nằm ở việc thay thế mọi engine khác, mà ở khả năng trở thành lớp truy vấn thống nhất phía trên nhiều nguồn dữ liệu mà không bắt buộc di chuyển dữ liệu vào một storage riêng.

2.2.6.1 Apache Spark vs Trino Apache Spark là engine xử lý dữ liệu phân tán tổng quát, mạnh ở batch ETL, data transformation, machine learning và streaming jobs. Đến các phiên bản Spark 3.5.x, Spark SQL đã có nhiều tối ưu cho Lakehouse như Adaptive Query Execution, statistics-based optimization, join strategy hints và Storage Partition Join với data source V2 như Iceberg. Tuy vậy, Spark vẫn phù hợp nhất khi truy vấn là một phần của pipeline xử lý dữ liệu lớn, nhiều bước, cần ghi dữ liệu hoặc cần fault tolerance theo lineage. Trino phù hợp hơn khi mục tiêu chính là phản hồi SQL tương tác, ad-hoc BI và truy vấn nhiều nguồn dữ liệu trong cùng một câu SQL.

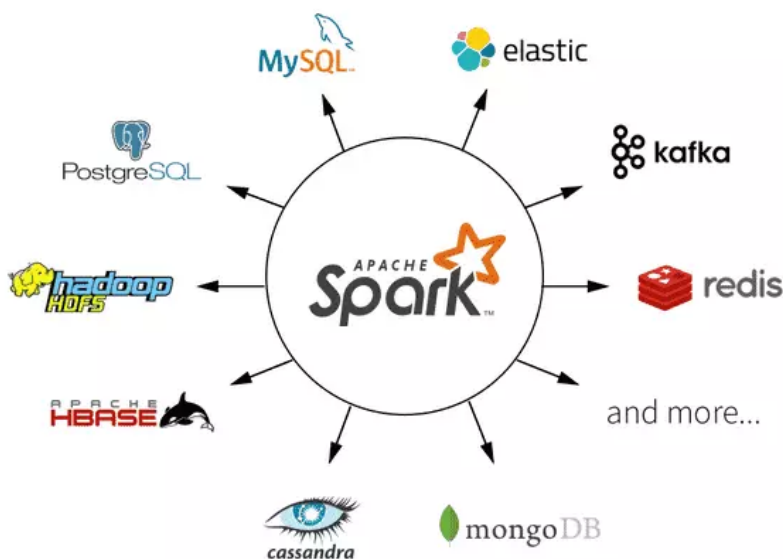


Figure 8: Apache Spark - Engine xử lý dữ liệu phân tán

Table 1: So sánh Apache Spark và Trino

Tiêu chí	Apache Spark	Trino
Mục tiêu thiết kế	Engine xử lý dữ liệu tổng quát cho batch, streaming, ML, ETL và Spark SQL.	Distributed SQL query engine cho ad-hoc analytics, interactive BI và federated query.
Mô hình thực thi	DAG-based execution, chia job thành stages/tasks; lineage giúp phục hồi lỗi và phù hợp job dài.	MPP, pipelined execution, coordinator lập kế hoạch và worker thực thi song song.
Tối ưu truy vấn hiện đại	Có AQE, runtime statistics, join hints, caching và các tối ưu data source V2/Storage Partition Join.	Có cost-based optimizer, dynamic filtering, predicate/projection pushdown, metadata/file-system caching và connector-level optimization.
Lakehouse workload	Rất mạnh cho ghi dữ liệu, ETL, compaction, transformation trên Iceberg/Delta/Hudi.	Rất mạnh cho đọc, phân tích tương tác và truy vấn Iceberg/Hive/object storage trực tiếp qua connector.
Hiệu năng truy vấn dữ liệu lớn	Rất tốt với batch query lớn, transformation nặng và pipeline có nhiều bước; latency có thể cao hơn với truy vấn ngắn do overhead lập lịch và shuffle.	Rất tốt với OLAP đọc nhiều dữ liệu, filter/join/aggregate trên Parquet/Iceberg, đặc biệt khi cần phản hồi nhanh cho ad-hoc SQL và BI.
Trường hợp phù hợp	Batch ETL, xử lý dữ liệu phức tạp, ML pipeline, streaming processing.	Ad-hoc SQL, federated query, BI dashboard, truy vấn đa nguồn không cần ETL trước.

2.2.6.2 Apache Hive vs Trino Apache Hive là lớp SQL-on-Hadoop truyền thống, ban đầu được thiết kế để đưa SQL vào hệ sinh thái Hadoop. Đến giai đoạn hiện đại, Hive không chỉ là MapReduce SQL: Hive có Tez, cost-based optimizer và LLAP để cải thiện độ trễ, cache và xử lý truy vấn ngắn. Tuy nhiên, Hive vẫn thường được xem là warehouse/metastore layer hoặc batch SQL engine hơn là lớp federated query cloud-native. Trong nhiều kiến trúc Lakehouse, Hive Metastore vẫn rất quan trọng, nhưng Trino thường được đặt phía trên để phục vụ truy vấn tương tác nhanh hơn.



Figure 9: Apache Hive - Lớp Data Warehouse cho hệ sinh thái Hadoop

Table 2: So sánh Apache Hive và Trino

Tiêu chí	Apache Hive	Trino
Mục tiêu thiết kế	Data warehouse trên Hadoop, SQL abstraction cho dữ liệu lưu trên HDFS/object storage và metadata Hive.	Query engine MPP cho SQL tương tác trên nhiều nguồn dữ liệu.
Mô hình thực thi	Phụ thuộc execution engine như Tez hoặc Spark; LLAP bổ sung daemon/cache cho truy vấn ngắn.	Engine thực thi riêng, pipelined và in-memory nhiều hơn, tối ưu cho latency thấp.
Vai trò metadata	Hive Metastore là thành phần metadata phổ biến trong hệ sinh thái Big Data.	Có thể dùng Hive Metastore như một catalog, nhưng không bị giới hạn bởi Hive.
Khả năng truy vấn đa nguồn	Tập trung chủ yếu vào bảng trong Hadoop/Hive ecosystem; không phải trọng tâm federated SQL.	Connector architecture phong phú: Iceberg, Hive, PostgreSQL, Kafka, object storage và nhiều hệ khác.
Hiệu năng truy vấn dữ liệu lớn	Phù hợp batch SQL trên dữ liệu rất lớn; Tez/LLAP giúp cải thiện latency nhưng vẫn kém linh hoạt hơn cho ad-hoc/federated BI hiện đại.	Nhanh hơn cho interactive SQL nhờ MPP, pipelined execution và tối ưu đọc dữ liệu qua connector.
Trường hợp phù hợp	Batch reporting, hệ Hadoop/Hive legacy, quản lý metadata bảng.	Interactive analytics, federated query, BI trên Lakehouse hiện đại.

2.2.6.3 Apache Impala vs Trino Apache Impala là MPP SQL engine được xây dựng để giảm độ trễ truy vấn trong hệ sinh thái Hadoop và các storage tương thích như HDFS, HBase, Kudu, S3/ADLS. Impala gần Trino hơn Spark/Hive ở mục tiêu interactive SQL, nhưng thiên về cụm dữ liệu đã chuẩn hóa quanh Hive Metastore/Impala catalog và các daemon `impalad`. Trino linh hoạt hơn khi cần truy vấn đồng thời nhiều catalog và hệ thống không đồng nhất, đặc biệt trong môi trường Kubernetes/Lakehouse mở.



Figure 10: Apache Impala - Engine truy vấn SQL phân tán hiệu năng cao trên Hadoop

Table 3: So sánh Apache Impala và Trino

Tiêu chí	Apache Impala	Trino
Mục tiêu thiết kế	Low-latency SQL cho dữ liệu trong Hadoop-compatible ecosystem như Hive/HDFS/Kudu/S3.	Low-latency SQL trên nhiều nguồn dữ liệu, nhấn mạnh federated analytics.
Mô hình thực thi	MPP query engine với impalad; có thể co-located với DataNode hoặc chạy tách riêng để đọc HDFS/S3/ADLS.	MPP query engine, coordinator/worker tách biệt, đọc dữ liệu qua connector.
Độ gắn với hệ sinh thái	Gắn với Hive Metastore/Impala catalog và các triển khai Hadoop-compatible đã chuẩn hóa.	Linh hoạt hơn trong môi trường nhiều storage/catalog/source khác nhau.
Lakehouse workload	Phù hợp khi dữ liệu nằm trong stack Impala/Hive/Kudu hoặc object storage đã tích hợp với Impala.	Phù hợp với kiến trúc Lakehouse mở: Iceberg, object storage, REST catalog và multi-source.
Hiệu năng truy vấn dữ liệu lớn	Rất tốt cho low-latency SQL khi dữ liệu, catalog, statistics và cluster được tối ưu cho Impala.	Rất tốt khi truy vấn dữ liệu lớn phân tán trên nhiều nguồn, nhất là Lakehouse/object storage và federated source.
Trường hợp phù hợp	SQL tương tác trong Hadoop cluster đã chuẩn hóa trên Impala/Cloudera.	SQL tương tác và federated query trong nền tảng dữ liệu dị thể, cloud-native hoặc Kubernetes.

2.2.6.4 Apache Doris vs Trino Apache Doris là hệ OLAP MPP có columnar storage, vectorized execution, cost-based optimizer, materialized view và khả năng phục vụ dashboard concurrency cao. Đến năm 2026, Doris không chỉ là OLAP database nội bộ mà còn đẩy mạnh vai trò **Lakehouse query acceleration**: tạo external catalog tới Hive, Iceberg, Hudi, Paimon, JDBC/PostgreSQL, cache meta-data/data và dùng materialized view để tăng tốc truy vấn external table. Tuy nhiên, nếu đặt trong bài toán federated ad-hoc SQL thuần túy, Trino vẫn tự nhiên hơn vì connector architecture là trọng tâm thiết kế ban đầu.



Figure 11: Apache Doris - Kho dữ liệu OLAP hiện đại cho truy vấn thời gian thực

Table 4: So sánh Apache Doris và Trino trong vai trò Lakehouse Query Engine

Tiêu chí	Apache Doris	Trino
Mục tiêu thiết kế	MPP OLAP database/query engine cho phân tích tốc độ cao, high-concurrency serving và lakehouse acceleration.	Distributed SQL query engine cho truy vấn tương tác và đa nguồn trên dữ liệu tại chỗ.
Vai trò trong Lakehouse	Truy vấn Hive/Iceberg/Hudi/Paimon qua external catalog; có thể tăng tốc bằng cache và materialized view.	Truy vấn trực tiếp Lakehouse qua connector/catalog mà không yêu cầu nạp vào storage riêng.
Storage strategy	Có thể mạnh với columnar storage nội bộ, materialized view, cache và pre-computation cho workload ổn định.	Tách compute khỏi storage; dữ liệu có thể nằm ở Iceberg/MinIO/PostgreSQL/Kafka.
Federated query	Có external catalog và join nhiều catalog, nhưng thường tối ưu nhất khi dùng như serving/acceleration layer.	Là điểm mạnh cốt lõi: truy vấn liên kết nhiều nguồn trong cùng một câu SQL.
Hiệu năng truy vấn dữ liệu lớn	Rất mạnh cho OLAP serving, high concurrency, materialized view, cache acceleration và workload có mẫu truy vấn ổn định.	Mạnh cho scan/join/aggregate trên dữ liệu lớn tại nguồn, đặc biệt khi không muốn nạp dữ liệu vào hệ serving riêng.
Trường hợp phù hợp	BI serving concurrency cao, dashboard ổn định, workload cần tăng tốc bằng materialized view.	Ad-hoc analytics, exploratory SQL, truy vấn nhiều nguồn và Lakehouse mở trên Kubernetes.

2.2.6.5 Nhận xét chung Xét riêng về **hiệu năng truy vấn dữ liệu lớn**, không có engine nào luôn nhanh nhất trong mọi trường hợp. Kết quả phụ thuộc vào định dạng dữ liệu, kích thước file, partitioning, statistics, cache, network, số lượng concurrent users và mức độ tối ưu catalog. Với các truy vấn tương tác trên dữ liệu lớn, đặc biệt là truy vấn đọc nhiều, lọc, join và aggregation trên dữ liệu đã được tổ chức tốt theo Parquet/Iceberg, Trino thường có lợi thế nhờ mô hình MPP và pipelined execution: dữ liệu được chia thành splits, đọc song song bởi nhiều worker và kết quả trung gian được truyền qua memory/network thay vì luôn materialize ra đĩa.

Spark có thể xử lý khối lượng dữ liệu rất lớn và ổn định hơn với các job dài, nhiều bước transformation hoặc cần fault tolerance cao. Các tối ưu hiện đại như AQE, runtime statistics và Storage Partition Join giúp Spark SQL tốt hơn nhiều cho Lakehouse. Tuy nhiên, với truy vấn SQL tương tác lặp lại, Spark vẫn thường chịu overhead lớn hơn do cơ chế lập lịch DAG, khởi tạo job và quản lý shuffle. Vì vậy Spark phù hợp hơn khi mục tiêu là throughput của pipeline batch, còn Trino phù hợp hơn khi mục tiêu là giảm latency cho ad-hoc query và BI query.

Hive hiện đại có Tez, CBO và LLAP nên không nên chỉ xem là MapReduce SQL cũ. Dù vậy, vai trò mạnh nhất của Hive trong nhiều nền tảng hiện nay vẫn là warehouse/metastore và batch SQL trong hệ Hadoop/Hive legacy. Nếu yêu cầu chính là phản hồi nhanh, truy vấn liên kết nhiều nguồn và phục vụ BI trên Lakehouse mở, Trino thường phù hợp hơn.

Impala cũng là MPP engine nên có thể đạt hiệu năng tốt cho low-latency SQL trên dữ liệu Hadoop-compatible, bao gồm HDFS, Kudu và object storage khi cụm được cấu hình đồng bộ. Điểm khác biệt là Trino linh hoạt hơn trong truy vấn dữ liệu nằm ngoài một stack cụ thể, chẳng hạn Iceberg REST catalog, PostgreSQL hoặc Kafka. Do đó, trong môi trường Lakehouse mở và đa nguồn, hiệu năng thực tế của Trino thường đến từ cả execution engine lẫn khả năng tránh bước di chuyển dữ liệu.

Doris có lợi thế khi workload là OLAP serving có concurrency cao, truy vấn dashboard lặp lại, hoặc có thể tận dụng cache, materialized views, pre-computation và columnar storage nội bộ. Nếu dùng Doris như Lakehouse query engine qua external catalog, hiệu năng phụ thuộc nhiều vào metadata cache, data cache, khả năng rewrite vào materialized view và mức độ ổn định của mẫu truy vấn. Trino thường phù hợp hơn cho truy vấn dữ liệu tại chỗ trên nhiều nguồn, còn Doris phù hợp hơn khi cần serving layer tối ưu cho BI có mẫu truy vấn ổn định.



Trino được chọn trong đề án vì phù hợp nhất với mục tiêu **federated query** và **interactive analytics** trên Lakehouse. Spark mạnh hơn cho ETL và batch processing; Hive phù hợp cho hệ Hadoop/Hive legacy và metadata; Impala phù hợp khi workload nằm trong stack Hadoop-compatible đã chuẩn hóa; Doris đáng cân nhắc khi cần OLAP serving, acceleration layer và high-concurrency BI. Trong kiến trúc của nhóm, Trino là lựa chọn cân bằng nhất vì có thể truy vấn trực tiếp Iceberg, PostgreSQL, Kafka và các nguồn dữ liệu khác mà không cần di chuyển dữ liệu về một storage riêng.

3 Dữ liệu và Kiến trúc Hệ thống

3.1 Chuẩn bị Dữ liệu

Để đánh giá Trino trong nhiều kiểu workload khác nhau, nhóm chuẩn bị hai bộ dữ liệu chính: **theLook Ecommerce** phục vụ phân tích nghiệp vụ thương mại điện tử và **TPC-DS** phục vụ benchmark chuẩn hóa. Trong đó, Data Generator không được xem là một bộ dữ liệu riêng, mà là chương trình sinh dữ liệu Ecommerce để mô phỏng luồng sự kiện và giao dịch phát sinh theo thời gian.

3.1.1 theLook Ecommerce

theLook Ecommerce là bộ dữ liệu mô phỏng hoạt động bán lẻ trực tuyến, bao gồm người dùng, sản phẩm, đơn hàng, chi tiết đơn hàng, tồn kho và sự kiện người dùng. Bộ dữ liệu này phù hợp để mô phỏng các bài toán phân tích thương mại điện tử trong thực tế như doanh thu theo kênh bán, hiệu quả danh mục sản phẩm, hành vi xem sản phẩm, chuyển đổi từ sự kiện truy cập sang đơn hàng và tình trạng tồn kho.

- **Tên bộ dữ liệu:** TheLook eCommerce BigQuery Dataset.
- **Nguồn tham chiếu:** Kaggle - Mustafa Keser.
- **Đặc điểm:** Bao gồm 07 bảng dữ liệu chính (Users, Products, Orders, Order Items, Events, Distribution Centers, Inventory Items) với mối quan hệ thực thể phức tạp, phù hợp cho việc thử nghiệm các kiến trúc Data Lakehouse.

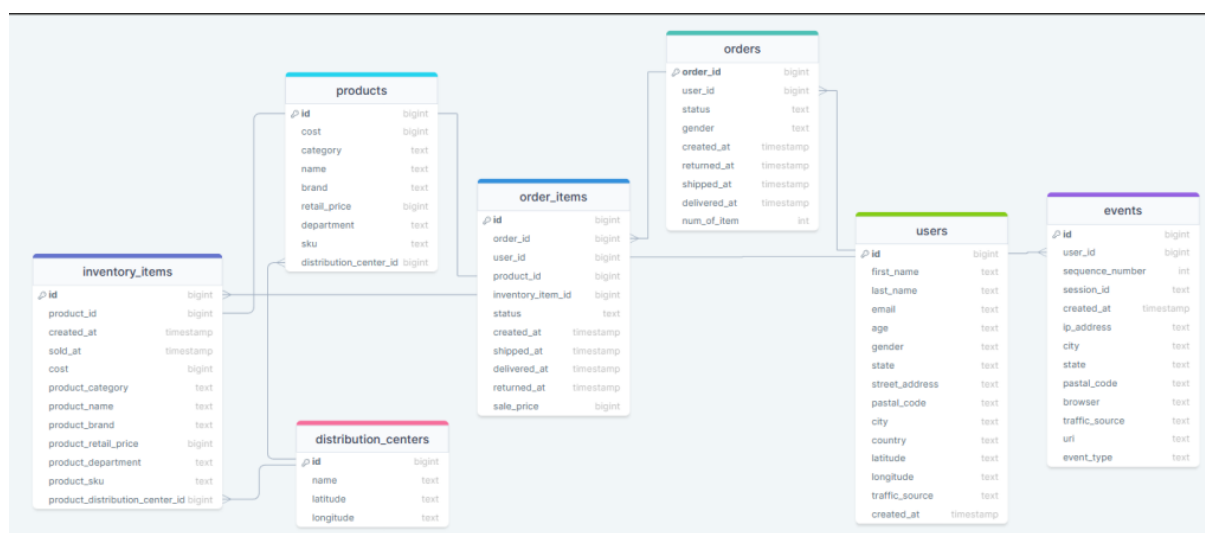


Figure 12: Sơ đồ quan hệ thực thể (ERD) của bộ dữ liệu TheLook eCommerce

Trong đồ án, theLook Ecommerce được dùng làm kịch bản chính cho **federated query** gần với môi trường công nghiệp. Thay vì giả định toàn bộ dữ liệu đã nằm trong một warehouse duy nhất, các nguồn dữ liệu có thể được phân tán theo vai trò:

- **Lakehouse/Iceberg:** Lưu dữ liệu đơn hàng, sản phẩm, inventory và các bảng phân tích đã qua xử lý.
- **PostgreSQL:** Mô phỏng nguồn nghiệp vụ/metadata vận hành như thông tin người dùng, cấu hình sản phẩm hoặc dữ liệu tham chiếu.
- **Kafka:** Mô phỏng luồng sự kiện hành vi người dùng như xem sản phẩm, thêm vào giỏ hàng, checkout.

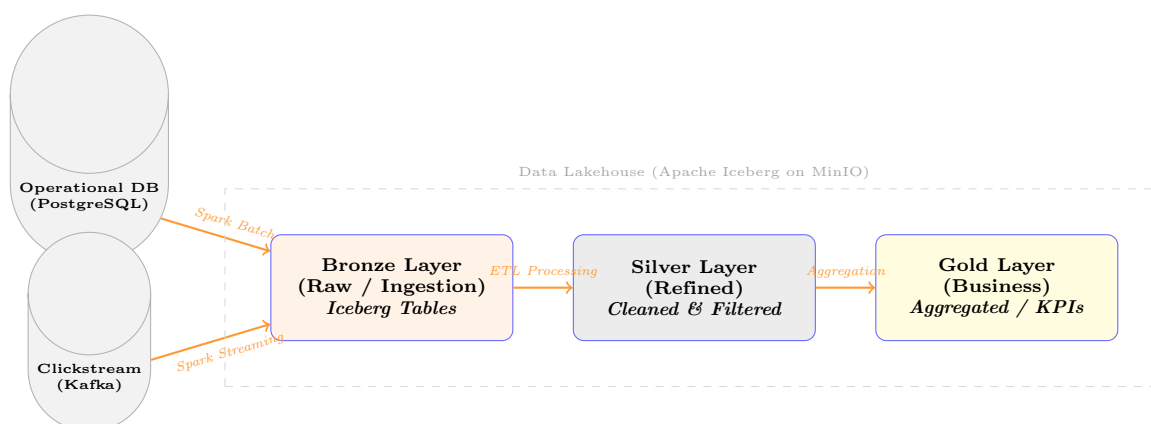
Với cách tổ chức này, Trino có thể thực hiện một câu SQL truy vấn đồng thời nhiều nguồn, ví dụ join doanh thu trong Iceberg với thông tin khách hàng ở PostgreSQL và sự kiện gần thời gian thực từ Kafka. Đây là mẫu bài toán phổ biến trong doanh nghiệp khi dữ liệu phân tán giữa hệ giao dịch, data lakehouse và hệ streaming, nhưng người phân tích vẫn cần một lớp SQL thống nhất để khai thác nhanh.

Trong đồ án, dữ liệu Ecommerce được sinh và mở rộng bằng **Data Generator** để tạo thêm các bản ghi giao dịch/sự kiện theo kịch bản kiểm và đảm bảo data phù hợp với các tác vụ incremental ETL và

streaming.

3.1.2 Kiến trúc Medallion và Data Lineage

Để quản lý vòng đời dữ liệu trên Lakehouse một cách khoa học, dự án áp dụng kiến trúc Medallion với ba tầng dữ liệu chính (Bronze, Silver, Gold) dựa trên định dạng Apache Iceberg. Mô hình này giúp đảm bảo tính tin cậy của dữ liệu qua từng giai đoạn xử lý và cung cấp khả năng truy vết (data lineage) xuyên suốt từ nguồn tới lớp phân tích.



Các tầng dữ liệu được định nghĩa như sau:

- **Bronze (Raw):** Lưu trữ dữ liệu thô từ các nguồn PostgreSQL và Kafka. Dữ liệu ở đây được giữ nguyên trạng thái gốc để phục vụ mục đích kiểm toán và tái xử lý khi cần thiết.
- **Silver (Refined):** Dữ liệu từ tầng Bronze được Spark xử lý để loại bỏ bản ghi trùng lặp, chuẩn hóa kiểu dữ liệu và áp dụng các ràng buộc schema. Đây là tầng dữ liệu tin cậy cho các tác vụ khám phá dữ liệu (Data Discovery).
- **Gold (Business):** Tầng cuối cùng chứa các bảng đã được tổng hợp theo các chiều kinh doanh (như doanh thu theo ngày, hành trình khách hàng). Dữ liệu tầng Gold được tối ưu hóa cho hiệu năng truy vấn của Trino và trực quan hóa trên Superset.

3.1.3 TPC-DS

Để đánh giá hiệu năng của hệ thống một cách khách quan, dự án sử dụng bộ chuẩn **TPC-DS (Transaction Processing Performance Council - Decision Support)**. Đây là tiêu chuẩn công nghiệp dùng để đo lường hiệu năng của các hệ thống hỗ trợ ra quyết định (Decision Support Systems) và phân tích dữ liệu lớn.

TPC-DS là bộ benchmark chuẩn cho các workload phân tích dữ liệu dạng decision support. Khác với theLook Ecommerce được dùng để mô phỏng kịch bản nghiệp vụ và federated query, TPC-DS được dùng cho mục tiêu **benchmark chuẩn hóa**: tạo cùng một schema, cùng một tập dữ liệu và cùng một tập truy vấn để so sánh hiệu năng giữa các engine một cách có thể tái lập.



Figure 13: Hội đồng TPC và các tập đoàn công nghệ thành viên xây dựng tiêu chuẩn đánh giá hiệu năng hệ thống dữ liệu

TPC-DS mô phỏng hệ thống bán lẻ đa kênh với các kênh store, catalog và web. Bộ benchmark này có đặc điểm gần với workload phân tích dữ liệu lớn trong doanh nghiệp: nhiều bảng fact/dimension, join nhiều chiều, aggregation, lọc theo thời gian, subquery, CTE và window functions. Vì vậy, TPC-DS phù hợp để đánh giá khả năng xử lý OLAP của Trino trên dữ liệu lớn và so sánh với Spark trong cùng điều kiện storage/catalog.

Trong đồ án, nhóm sử dụng TPC-DS để tạo dữ liệu và tập truy vấn chuẩn nhằm so sánh hiệu năng giữa Trino và Apache Spark:

- Dữ liệu gồm các bảng bán lẻ đa kênh như `store_sales`, `catalog_sales`, `web_sales`, `inventory`, `customer`, `date_dim`.
- Các truy vấn bao gồm nhiều dạng OLAP phổ biến: multi-join, aggregation, subquery, CTE và window functions.
- Dữ liệu benchmark được materialize vào Iceberg qua Hive Metastore để cả Trino và Spark cùng đọc từ một lớp storage/catalog thống nhất.
- Kết quả benchmark tập trung vào thời gian thực thi truy vấn, mức độ ổn định giữa các lần chạy và sự khác biệt giữa interactive query engine và batch processing engine.

3.2 Xây dựng Hệ thống

3.2.1 Tổng quan Nền tảng

Nhóm đã xây dựng một **K8s Data Platform (Lakehouse)** hoàn chỉnh trên Google Kubernetes Engine (GKE). Nền tảng này tích hợp các công nghệ mã nguồn mở hàng đầu vào một kiến trúc Lakehouse nhất quán.

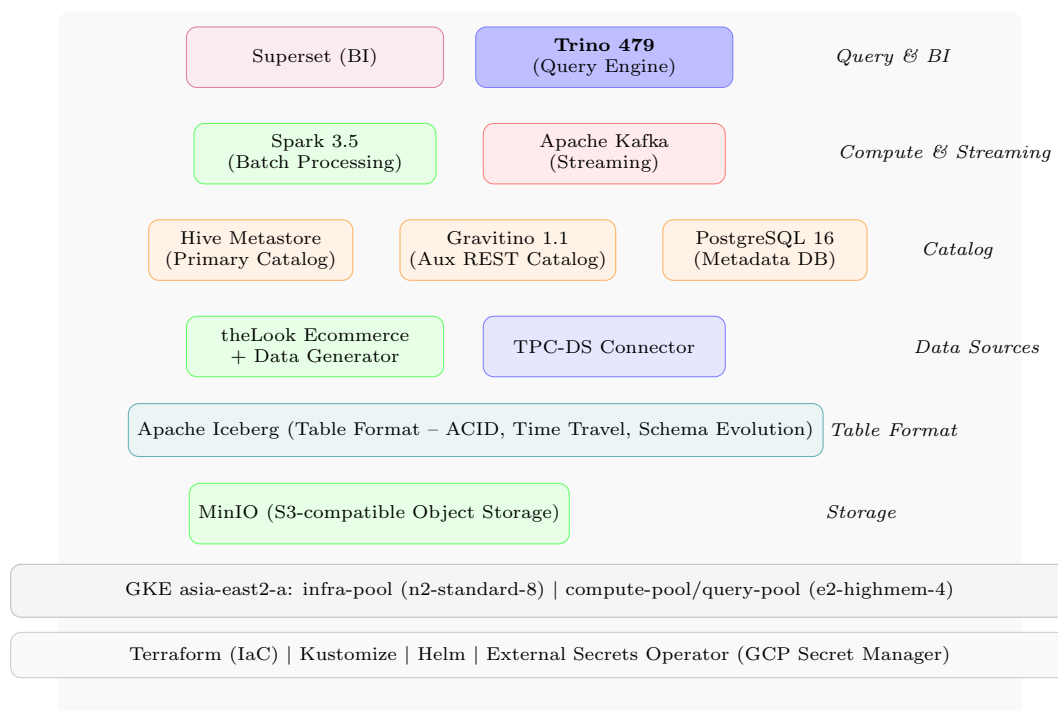


Figure 14: Kiến trúc tổng thể của K8s Data Platform (Lakehouse)

3.2.2 Cơ sở Hạ tầng (Infrastructure Layer)

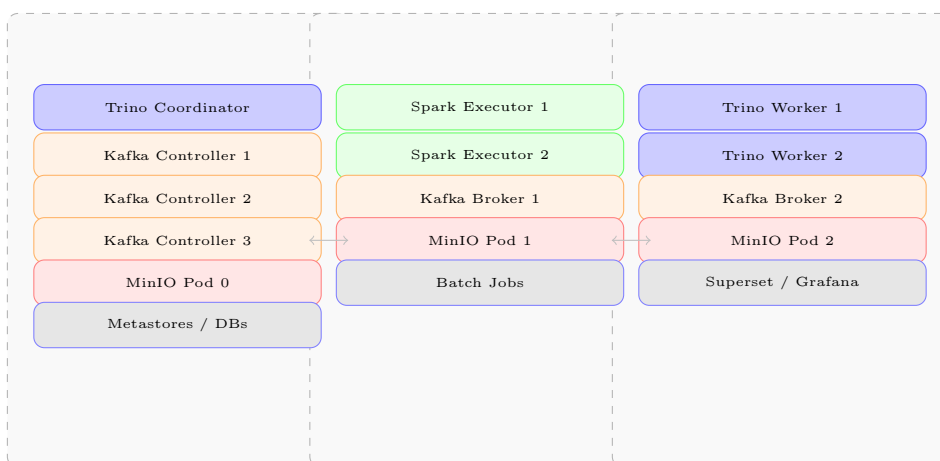
3.2.2.1 Google Kubernetes Engine (GKE) Cluster GKE được triển khai dạng zonal cluster tại asia-east2-a thuộc region asia-east2 với các đặc điểm:

- **Private cluster:** Worker nodes không có external IP, outbound qua Cloud NAT.
- **VPC-Native networking:** Pod và Service CIDR riêng biệt.
- **Dataplane V2:** Sử dụng datapath nâng cao của GKE cho network performance và security.
- **Workload Identity:** Pods xác thực GCP API không cần SA key file.

3.2.2.2 Node Pool Strategy Để đảm bảo tính sẵn sàng cao và tối ưu hóa tài nguyên, các thành phần cốt lõi của hệ thống được triển khai theo mô hình Cluster phân tán trên các Node Pool của Kubernetes:

Table 5: Cấu hình Node Pool

Pool	Machine Type	Label	Workloads
infra-pool	n2-standard-8	role=infra	PostgreSQL, Gravitino, Hive Metastore, Kafka controllers, Trino Coordinator
compute-pool	e2-highmem-4	role=compute	Spark jobs/executors, Kafka brokers, MinIO pods
query-pool	e2-highmem-4	role=query	Trino Workers , Superset, Kafka brokers, MinIO pods



✓ data-platform-cluster

⚠ GKE can't perform maintenance because the Pod Disruption Budget allows for 0 Pod evictions. Update the Pod Disruption Budget.

[Overview](#) [Details](#) [Nodes](#) [Storage](#) [Observability](#) [Logs](#) [App Errors \(99+\)](#)

Status Running	Mode Standard	Number of nodes 3	Control plane zone asia-east2-a	Endpoint 34.96.141.82
-------------------	------------------	----------------------	------------------------------------	--------------------------

Optimize your GKE experience ▼

Cluster overview

Name	data-platform-cluster
Fleet	None
Location type	Zonal
Control plane zone	asia-east2-a
Default node zones ?	asia-east2-a ✎
Release channel ?	Regular channel ✎
Version ?	1.35.3-gke.1234000 ✎ Upgrade available
Labels	None ✎

Resources

[Next step: Deploying workloads](#)

[View all Kubernetes documentation](#)

[Explore tutorials](#)

Figure 15: Giao diện của quản lý cluster GKE của platform

Node pools [Create user-managed node pool](#)

Name	Status	Version	Number of nodes	Machine type	Image type	Autoscaling	Default IPv4 Pod IP address range
compute-pool	Ok	1.35.3-gke.1234000	1	e2-highmem-4	Container-Optimized OS with containerd (cos_containerd)	0 - 1 nodes per zone	10.20.0.0/16
infra-pool	Ok	1.35.3-gke.1234000	1	n2-standard-8	Container-Optimized OS with containerd (cos_containerd)	0 - 1 nodes per zone	10.20.0.0/16
query-pool	Ok	1.35.3-gke.1234000	1	e2-highmem-4	Container-Optimized OS with containerd (cos_containerd)	0 - 1 nodes per zone	10.20.0.0/16

Nodes

Name	Status	Node type	CPU requested	CPU allocatable	Memory requested	Memory allocatable	Storage requested	Storage allocatable
gke-data-platform-cluster-compute-pool-695aa0ef-47zx	Ready	User-managed	3 CPU	3.92 CPU	12.76 GB	29.77 GB	0 B	0 B
gke-data-platform-cluster-infra-pool-8125a22b-tjrd	Ready	User-managed	4.75 CPU	7.91 CPU	17.7 GB	29.77 GB	0 B	0 B
gke-data-platform-cluster-query-pool-24615428-fvx9	Ready	User-managed	2.7 CPU	3.92 CPU	16.04 GB	29.77 GB	0 B	0 B

Figure 16: Thông tin các nodes và pools của cluster

```
PS D:\Projects\k8s-data-platform> kubectl get nodes
NAME                                STATUS    ROLES    AGE     VERSION
gke-data-platform-cluster-compute-pool-695aa0ef-47zx Ready    <none>   2d8h    v1.35.3-gke.1234000
gke-data-platform-cluster-infra-pool-8125a22b-tjrd Ready    <none>   2d21h    v1.35.3-gke.1234000
gke-data-platform-cluster-query-pool-24615428-fvx9 Ready    <none>   2d10h    v1.35.3-gke.1234000
```

Figure 17: Kết quả lệnh `kubectl get nodes` ở local

```
PS D:\Projects\k8s-data-platform> kubectl get pods -n data-platform
NAME                                READY    STATUS    RESTARTS   AGE
benchmark-data-helper               1/1      Running   1 (7m29s ago)  67m
data-platform-kafka-broker-pool-0  1/1      Running   0           2d10h
data-platform-kafka-broker-pool-1  1/1      Running   0           2d10h
data-platform-kafka-controller-pool-2 1/1      Running   0           2d10h
data-platform-kafka-controller-pool-3 1/1      Running   0           2d21h
data-platform-kafka-controller-pool-4 1/1      Running   0           2d21h
data-platform-kafka-entity-operator-77df597484-vv9sn 2/2      Running   7 (2d10h ago)  2d10h
gravitino-849bc77f7f-8rkjw         1/1      Running   0           2d21h
hive-metastore-0                    1/1      Running   0           2d21h
kafka-exporter-694b5874b4-4t8bj      1/1      Running   0           2d8h
minio-0                              1/1      Running   0           2d10h
minio-1                              1/1      Running   0           2d10h
minio-2                              1/1      Running   0           2d10h
postgres-0                           1/1      Running   0           2d21h
postgres-source-698db4779b-wjj7k    1/1      Running   0           2d10h
redis-0                               1/1      Running   0           2d8h
redis-exporter-7b5d565f5c-fvj5r      1/1      Running   0           2d8h
superset-6db5bc7d6d-bz64h            1/1      Running   0           2d10h
trino-coordinator-5b7875fc8b-4pgcz   0/1      Running   0           14s
trino-coordinator-6b9cdc9fcf-2cq1c   1/1      Running   0           2m3s
trino-worker-5f9df67b-2mpj8          1/1      Running   0           7h8m
trino-worker-5f9df67b-66vrl          1/1      Running   0           7h8m
trino-worker-77bbdfb4d8-szf2p        0/1      Pending   0           14s
```

Figure 18: Kết quả lệnh `kubectl get pods` ở local

Ba node pool đều cấu hình autoscaling từ 0 đến 1 node và sử dụng **Spot VMs** để tối ưu chi phí cho môi trường phát triển. Mỗi node dùng đĩa `pd-balanced` dung lượng 50GiB, bật Shielded VM, auto-repair và auto-upgrade. Các workload được ghim vào node pool bằng `nodeSelector`, `nodeAffinity` và `tolerations` tương ứng với nhãn role.

3.2.2.3 Quản lý Secrets

Secrets được quản lý theo chuỗi:

GCP Secret Manager $\xrightarrow{\text{ESO}}$ K8s Secret $\xrightarrow{\text{envFrom}}$ Pod

External Secrets Operator (ESO) sử dụng Workload Identity để đồng bộ secrets, không lưu credentials trong Git.

3.2.2.4 Phạm vi cấu hình được trình bày Báo cáo tập trung vào các thành phần trực tiếp tạo nên nền tảng lakehouse và query engine: GKE, Terraform, Kustomize, External Secrets Operator, PostgreSQL, MinIO, Gravitino, Hive Metastore, Spark, Kafka, Trino, Superset và Ingress. Các công cụ phụ trợ như Redis, Grafana và Airflow có thể tồn tại trong manifest/Helm values để phục vụ vận hành, nhưng không được đưa vào phần kiến trúc chính của đồ án.

3.2.3 Storage và Catalog Layer

3.2.3.1 MinIO – Object Storage (Distributed Cluster) MinIO đóng vai trò là data lake storage, được triển khai theo mô hình **Distributed Cluster** sử dụng StatefulSet với 3 replicas để đảm bảo tính sẵn sàng cao và khả năng chịu lỗi:

- **Erasure Coding:** Cụm MinIO được cấu hình với 3 nodes, cho phép hệ thống vẫn hoạt động và bảo toàn dữ liệu ngay cả khi có node bị lỗi. Dữ liệu được chia thành các data chunks và parity chunks trải rộng trên các ổ đĩa của 3 replicas.
- **High Availability:** Triển khai dưới dạng StatefulSet giúp duy trì định danh mạng ổn định (minio-0, minio-1, minio-2) và tự động gắn lại các Persistent Volume khi pod khởi động lại.
- Cung cấp S3-compatible API cho Spark, Trino, Iceberg.
- Mỗi replica dùng volumeClaimTemplates dung lượng 20Gi với StorageClass `balanced-rwo` trên GCE Persistent Disk.
- Service nội bộ minio mở port 9000 cho S3 API và 9001 cho console.
- Dữ liệu được tổ chức theo cấu trúc: `s3a://iceberg/lakehouse/{catalog}/{schema}/{table}/`.

3.2.3.2 Apache Iceberg – Table Format Apache Iceberg cung cấp các tính năng enterprise cho data lake:

- **ACID transactions:** Đảm bảo tính nhất quán khi đọc/ghi đồng thời.
- **Time Travel:** Truy vấn dữ liệu tại bất kỳ thời điểm nào trong lịch sử.
- **Schema Evolution:** Thêm/xóa/đổi tên cột không cần rewrite toàn bộ dữ liệu.
- **Partition Evolution:** Thay đổi chiến lược partitioning mà không ảnh hưởng đến queries cũ.

3.2.3.3 Hive Metastore – Catalog chính Hive Metastore là catalog chính của nền tảng trong phạm vi pipeline và benchmark. Các bảng Bronze/Silver/Gold và dữ liệu TPC-DS được ghi dưới dạng Iceberg table trên MinIO, nhưng metadata bảng được quản lý qua Hive Metastore:

- StatefulSet 1 replica, image `apache/hive:3.1.3`, chạy trên `role=infra`.
- Expose Thrift endpoint `thrift://hive-metastore:9083`.
- Trino và Spark dùng catalog `iceberg_hive` của Hive để tương tác với Lakehouse là chính, việc sử dụng thêm Gravitino để kết nối với Hive và nhiều nguồn dữ liệu khác giúp mở ra khả năng tương tác với nhiều kho lưu trữ cho các engine.
- Metadata lưu trong PostgreSQL; cấu hình `hive-site.xml` được render từ Secret khi pod khởi động.

3.2.3.4 Gravitino – Catalog phụ cho truy vấn nâng cao Apache Gravitino được triển khai như một Iceberg REST Catalog phụ. Thành phần này không phải đường chính của pipeline/benchmark hiện tại, mà phục vụ các kịch bản mở rộng như quản lý nhiều catalog, kết nối metadata từ nhiều nguồn và thử nghiệm truy vấn nâng cao:

- REST API endpoint: `http://gravitino:9001/iceberg/`.
- Backend: PostgreSQL `catalog_metastore_db`.
- S3 FileIO: Gravitino có thể tự xử lý đọc/ghi Parquet trên MinIO.

- Deployment chạy 1 replica trên `role=infra`; init containers chờ PostgreSQL/MinIO và render file cấu hình từ Secret.

3.2.3.5 PostgreSQL – Metadata Store PostgreSQL 16 được dùng làm cơ sở dữ liệu metadata cho Gravitino và Hive Metastore:

- StatefulSet 1 replica, PVC `postgres-pvc` dung lượng 20Gi.
- Service nội bộ `postgres:5432`; secret được đồng bộ qua ESO.
- Resource request/limit lần lượt là 200m/2 CPU và 1Gi/4Gi memory.

3.2.4 Streaming và Processing Layer

3.2.4.1 Apache Kafka – Streaming Kafka được triển khai bằng Strimzi Operator theo chế độ KRaft, tách controller và broker thành hai node pool logic:

- Kafka version 4.1.1, metadata version 4.1-IV0.
- Controller pool gồm 3 replica, mỗi replica dùng PVC 5Gi, chạy trên `role=infra`.
- Broker pool gồm 2 replica, mỗi replica dùng PVC 20Gi, chạy trên `role=compute` hoặc `role=query`.
- Listener nội bộ `data-platform-kafka-kafka-bootstrap:9092`; topic mẫu `click-events` có 3 partitions, 2 replicas và retention 7 ngày.

3.2.4.2 Apache Spark – Batch Processing Spark chạy trên Kubernetes thông qua Spark Operator. Các SparkApplication trong thư mục `pipelines/` dùng ServiceAccount `spark-driver`, RBAC nội bộ namespace và cấu hình chung từ `spark-configmap`:

- Catalog chính cho pipeline: Iceberg catalog kiểu Hive, trỏ tới `thrift://hive-metastore:9083`. Trong benchmark catalog này được đặt tên `iceberg_hive`; trong một số pipeline SparkApplication được đặt tên `catalog_iceberg` (REST Catalog của Gravitino) nhưng backend vẫn là Hive Metastore.
- Catalog phụ thử nghiệm: Gravitino REST tại `http://gravitino:9001/iceberg/`, dùng cho kịch bản quản lý nhiều catalog/truy vấn nâng cao.
- MinIO được truy cập bằng S3A/S3FileIO, credentials lấy từ `spark-minio-secret`.
- Có package `spark-sql-kafka-0-10_2.12:3.5.5` để đọc/ghi Kafka trong các pipeline streaming.

3.2.5 Triển khai Trino trên Kubernetes

3.2.5.1 Helm Chart Configuration Trino được triển khai bằng Official Helm Chart với các tham số quan trọng:

```
1 image:
2   repository: trinodb/trino
3   tag: "479"
4
5 server:
6   workers: 2
7
8 coordinator:
9   jvm:
10    maxHeapSize: "3G"
11   resources:
12    requests:
13     memory: "3.5Gi"
14     cpu: "200m"
15    limits:
16     memory: "6Gi"
17     cpu: "2"
18   nodeSelector:
19    role: infra    # Coordinator trên Spot node
20
```

```
21 worker:
22   jvm:
23     maxHeapSize: "8G"
24   resources:
25     requests:
26       memory: "9Gi"
27       cpu: "1"
28     limits:
29       memory: "12Gi"
30       cpu: "4"
31   affinity:
32     nodeAffinity:
33       requiredDuringSchedulingIgnoredDuringExecution:
34         nodeSelectorTerms:
35           - matchExpressions:
36             - key: role
37               operator: In
38               values: [query, compute]
39   tolerations:
40     - key: "cloud.google.com/gke-spot"
41       operator: "Equal"
42       value: "true"
43       effect: "NoSchedule"
```

Listing 1: Cấu hình Helm values cho Trino (dev.yaml)

3.2.5.2 Catalog Configuration Catalog chính của Trino cho pipeline và benchmark là iceberg_hive, kết nối Iceberg với Hive Metastore và MinIO:

```
1 connector.name=iceberg
2 iceberg.catalog.type=hive_metastore
3 hive.metastore.uri=thrift://hive-metastore:9083
4
5 fs.native-s3.enabled=true
6 s3.endpoint=http://minio:9000
7 s3.aws-access-key=${ENV:MINIO_ACCESS_KEY}
8 s3.aws-secret-key=${ENV:MINIO_SECRET_KEY}
9 s3.path-style-access=true
10 s3.region=us-east-1
```

Listing 2: iceberg_hive.properties

Ngoài catalog Hive Metastore chính, Helm values hiện tại còn cấu hình các catalog sau:

- **catalog_iceberg**: Iceberg REST catalog dùng Gravitino tại `http://gravitino:9001/iceberg/`, phục vụ thử nghiệm catalog phụ và truy vấn nâng cao.
- **hive**: Hive connector đọc dữ liệu qua Hive Metastore và MinIO.
- **kafka**: Kafka connector trỏ tới `data-platform-kafka-kafka-bootstrap:9092`, đọc mô tả bảng từ `/etc/trino/schemas`.
- **postgres_source**: PostgreSQL connector trỏ tới database nguồn `thelook_db`.

3.2.6 BI và Truy cập Hệ thống

3.2.6.1 Apache Superset Superset được dùng làm lớp BI để kết nối tới Trino, tạo dashboard và kiểm thử truy vấn tương tác:

- Deployment 1 replica trên node `role=query`.
- Service nội bộ `superset:8088`, dữ liệu ứng dụng lưu trên PVC `superset-pvc` dung lượng 5Gi.
- Secret key lấy từ `superset-secret`; init script được mount từ ConfigMap `superset-init-config`.

3.2.6.2 Ingress Ingress data-platform-ingress dùng ingressClassName=cilium để expose các endpoint cần dùng trong báo cáo:

- trino.data.local trở tới service trino:8080.
- superset.data.local trở tới service superset:8088.

3.2.7 Tóm tắt Cấu hình Triển khai

Table 6: Các thành phần chính đang được cấu hình trong cluster

Thành phần	Kiểu triển khai	Vị trí	Vai trò
PostgreSQL 16	StatefulSet 1 replica	role=infra	Metadata DB cho Gravitino và Hive Metastore
MinIO	StatefulSet 3 replica	Trải trên các node	Object storage S3-compatible cho Iceberg/Parquet
Gravitino 1.1.0	Deployment 1 replica	role=infra	Catalog phụ qua Iceberg REST, phục vụ truy vấn nâng cao/nhiều catalog
Hive Metastore 3.1.3	StatefulSet 1 replica	role=infra	Catalog chính cho pipeline và benchmark qua Thrift 9083
Kafka 4.1.1	Strimzi KRaft	Infra + compute/query	Streaming layer cho dữ liệu sự kiện
Spark Operator	Helm chart	data-platform	Chạy SparkApplication cho Bronze/Silver/Gold pipelines
Trino 479	Helm chart, 1 coordinator + 2 workers	Infra + compute/query	Distributed SQL query engine, federation qua catalog
Superset	Deployment 1 replica	role=query	BI/dashboard truy cập qua Trino

3.2.7.1 Local Development – Docker Compose Để phát triển cục bộ, nhóm sử dụng Docker Compose với cấu hình tối ưu cho máy local (16GB RAM):

```

1 -server
2 -Xmx2G
3 -XX:+UseG1GC
4 -XX:G1HeapRegionSize=32M

```

Listing 3: Cấu hình Trino cho local (jvm.config)

Và các memory limits:

- query.max-memory-per-node=768MB
- query.max-memory=768MB
- query.max-total-memory=1536MB

4 Hiện thực

Phần hiện thực tập trung vào hai khả năng cốt lõi của Trino trong nền tảng dữ liệu lớn: truy vấn đa nguồn dữ liệu thông qua connector và thực thi truy vấn phân tán theo mô hình MPP. Các ví dụ dưới đây minh họa cách nhóm cấu hình, kết nối và khai thác Trino trên dữ liệu Iceberg, PostgreSQL và Kafka.

4.1 Truy vấn Đa nguồn

Trino cho phép người dùng truy vấn nhiều hệ thống lưu trữ khác nhau bằng cùng một cú pháp SQL. Trong đồ án, nhóm dùng `iceberg_hive` làm catalog chính cho lakehouse storage qua Hive Metastore. Catalog `catalog_iceberg` qua Gravitino REST được giữ như catalog phụ cho các kịch bản truy vấn nâng cao và kết nối nhiều nguồn metadata. Bên cạnh đó, Trino còn có `hive`, `postgres_source` và `kafka` để truy vấn dữ liệu quan hệ và streaming.

```
1 connector.name=postgresql
2 connection-url=jdbc:postgresql://postgres-source:5432/thelook_db
3 connection-user=db_user
4 connection-password=db_password
```

Listing 4: Cấu hình PostgreSQL catalog

```
1 connector.name=kafka
2 kafka.nodes=data-platform-kafka-kafka-bootstrap:9092
3 kafka.table-description-dir=/etc/trino/schemas
4 kafka.hide-internal-columns=false
```

Listing 5: Cấu hình Kafka catalog

```
1 connector.name=hive
2 hive.metastore.uri=thrift://hive-metastore:9083
3 fs.native-s3.enabled=true
4 s3.endpoint=http://minio:9000
5 s3.aws-access-key=${MINIO_ACCESS_KEY}
6 s3.aws-secret-key=${MINIO_SECRET_KEY}
7 s3.path-style-access=true
```

Listing 6: Cấu hình Hive catalog

```
1 connector.name=iceberg
2 iceberg.catalog.type=rest
3 iceberg.rest-catalog.uri=http://gravitino:9001/iceberg/
4 fs.native-s3.enabled=true
5 s3.endpoint=http://minio:9000
6 s3.aws-access-key=${MINIO_ACCESS_KEY}
7 s3.aws-secret-key=${MINIO_SECRET_KEY}
8 s3.path-style-access=true
```

Listing 7: Cấu hình Gravitino REST catalog

Với các catalog này, một câu SQL có thể kết hợp dữ liệu vận hành trong PostgreSQL, dữ liệu lịch sử trong Iceberg và sự kiện mới phát sinh từ Kafka:

```
1 SELECT
2     o.user_id,
3     u.email,
4     count(e.id) as total_views_in_lakehouse,
5     count(distinct o.order_id) as total_orders_in_postgres
6 FROM postgres_source.demo.orders o
7 JOIN postgres_source.demo.users u ON o.user_id = u.id
8 JOIN iceberg_hive.bronze.events e ON o.user_id = e.user_id
9 WHERE e.event_type = 'product'
10 GROUP BY o.user_id, u.email
11 HAVING count(distinct o.order_id) > 1
```

```
12 ORDER BY total_views_in_lakehouse DESC
13 LIMIT 10;
```

Listing 8: Truy vấn liên kết dữ liệu Iceberg và PostgreSQL

```
trino> SELECT
->   o.user_id,
->   u.email,
->   count(e.id) as total_views_in_lakehouse,
->   count(distinct o.order_id) as total_orders_in_postgres
-> FROM postgres_source.demo.orders o
-> JOIN postgres_source.demo.users u ON o.user_id = u.id
-> JOIN iceberg_hive.bronze.events e ON o.user_id = e.user_id
-> WHERE e.event_type = 'product'
-> GROUP BY o.user_id, u.email
-> HAVING count(distinct o.order_id) > 1
-> ORDER BY total_views_in_lakehouse DESC
-> LIMIT 10;
```

user_id	email	total_views_in_lakehouse	total_orders_in_postgres
199	joanne.smith@example.net	8	2
855	angie.mckinney@example.org	6	2
916	heather.rodriquez@example.net	6	2
159	sarah.dominguez@example.com	6	2
191	ian.robinson@example.org	4	2
568	ellen.cannon@example.org	4	2
442	pamela.sanchez@example.org	2	2

(7 rows)

Query 20260508_144121_00012_pk4e5, FINISHED, 2 nodes
Splits: 34 total, 34 done (100.00%)
4.09 [15.5K rows, 401KiB] [3.78K rows/s, 98.1KiB/s]

Figure 19: Kết quả truy vấn trả về từ Trino

```
1 SELECT user_id, event_type, browser, created_at
2 FROM kafka.default.click_events
3 LIMIT 5
```

Listing 9: Truy vấn trực tiếp dữ liệu streaming từ Kafka

```
trino> SELECT user_id, event_type, browser, created_at FROM kafka.default.click_events LIMIT 5;
```

user_id	event_type	browser	created_at
611	product	Chrome	2026-05-08 14:27:36.590408
611	home	Chrome	2026-05-08 14:27:59.590408
611	cart	Chrome	2026-05-08 14:28:18.590408
611	purchase	Chrome	2026-05-08 14:29:22.590408
413	department	IE	2026-05-08 14:27:37.674326

(5 rows)

Query 20260508_143719_00008_pk4e5, FINISHED, 2 nodes
Splits: 6 total, 6 done (100.00%)
0.27 [6.36K rows, 2.11MiB] [23.2K rows/s, 7.71MiB/s]

```
trino> SELECT count(*) FROM kafka.default.click_events;
_col0
-----
6361
(1 row)
```

Query 20260508_143732_00009_pk4e5, FINISHED, 2 nodes
Splits: 6 total, 6 done (100.00%)
0.76 [6.36K rows, 2.11MiB] [8.39K rows/s, 2.79MiB/s]

Figure 20: Kết quả truy vấn trả về từ Trino

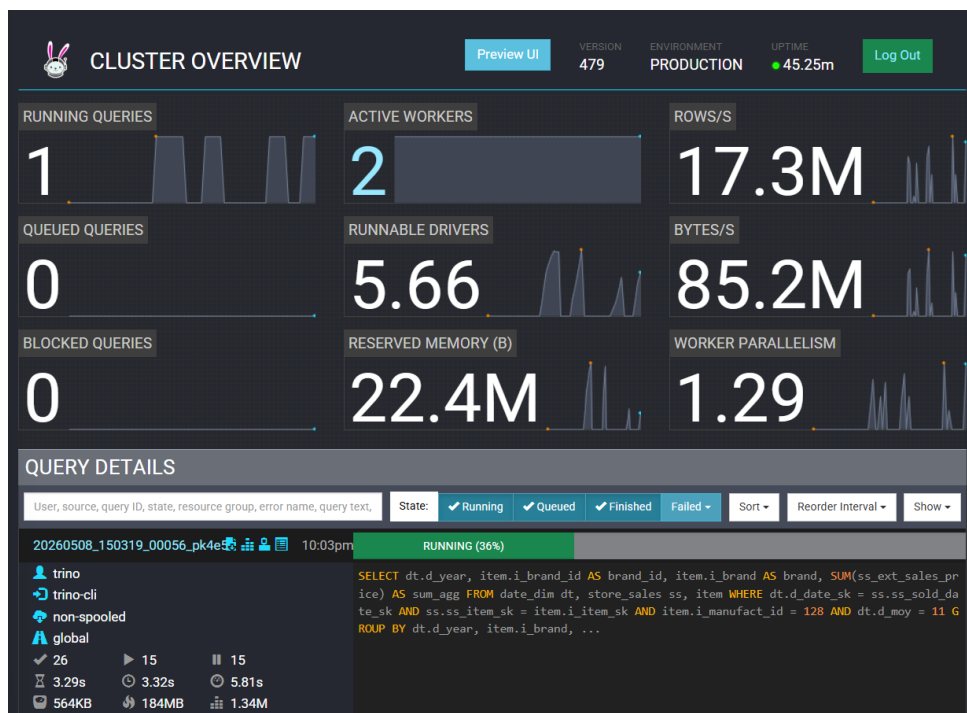
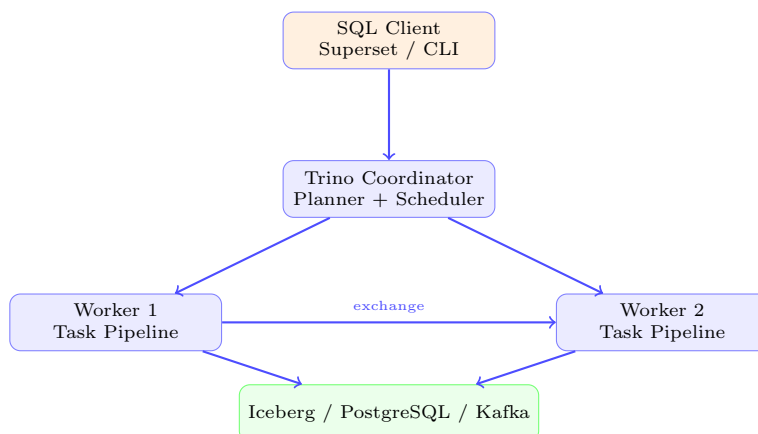


Figure 21: Giao diện của Trino Cluster

Cách tiếp cận này giúp giảm nhu cầu sao chép dữ liệu về một hệ thống trung gian chỉ để phân tích ad-hoc. Trino đóng vai trò lớp truy vấn thống nhất, còn dữ liệu vẫn nằm tại nguồn phù hợp nhất với vòng đời của nó.

4.2 Truy vấn Phân tán

Khi nhận một truy vấn, Trino coordinator phân tích cú pháp, tối ưu kế hoạch thực thi, chia truy vấn thành các stage và phân phối task xuống các worker. Mỗi worker xử lý một phần dữ liệu, thực hiện scan, filter, projection, join hoặc aggregation cục bộ, sau đó trao đổi dữ liệu qua mạng để tạo kết quả cuối cùng.



Ví dụ dưới đây là một truy vấn phân tích doanh thu trên bảng Iceberg. Trino có thể đẩy xuống các thao tác lọc cột, đọc Parquet theo split, gom nhóm cục bộ tại worker và chỉ shuffle phần dữ liệu cần thiết cho aggregation cuối.

```

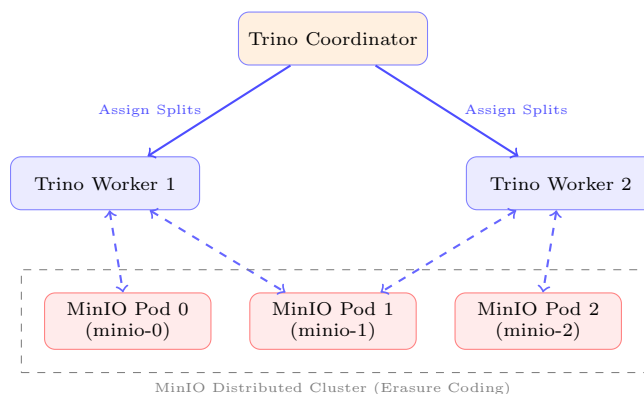
1 SELECT
2     order_date ,
3     product_category ,
4     SUM(total_revenue) AS revenue ,
5     SUM(total_items_sold) AS items_sold
6 FROM iceberg_hive.gold.sales_performance
  
```

```
7 WHERE order_date >= DATE '2025-07-01'
8 GROUP BY order_date, product_category
9 ORDER BY order_date, revenue DESC;
```

Listing 10: Truy vấn phân tích trên bảng Iceberg

Trong quá trình triển khai, nhóm cấu hình riêng coordinator và worker để phù hợp với vai trò của từng thành phần. Coordinator ưu tiên độ ổn định để lập lịch truy vấn, còn worker được đặt trên node pool dành cho query workload để mở rộng năng lực xử lý khi cần.

Để tối ưu hóa hiệu năng cho Data Lakehouse, hệ thống tận dụng khả năng đọc song song từ Trino Worker tới cụm MinIO được triển khai dưới dạng StatefulSet với 3 replicas. Thay vì đọc qua một điểm tập trung, các Trino Worker sẽ thiết lập các kết nối đồng thời tới các pod MinIO (*minio-0*, *minio-1*, *minio-2*) để quét các file Parquet/Orc được phân mảnh (splits).



Cơ chế này giúp loại bỏ nút thắt cổ chai tại lớp lưu trữ. Khi một truy vấn quét bảng Iceberg lớn, Trino Coordinator sẽ chia bảng thành hàng ngàn splits, sau đó các Worker sẽ kéo dữ liệu trực tiếp từ các node MinIO chứa chunk dữ liệu đó, tận dụng tối đa băng thông mạng nội bộ của Kubernetes cluster.

4.3 Tích hợp với Superset trong trực quan hóa dữ liệu

Apache Superset được triển khai làm lớp Business Intelligence (BI) để trực quan hóa toàn bộ hệ thống dữ liệu từ Lakehouse. Bằng cách kết nối trực tiếp với Trino Query Engine thông qua giao thức SQLAlchemy, Superset cho phép chuyển đổi các truy vấn SQL phức tạp thành các biểu đồ tương tác thời gian thực.

Việc tích hợp này mang lại các lợi điểm chính:

- **Truy vấn tương tác:** Tận dụng tốc độ của Trino để phản hồi nhanh các thao tác lọc và phân tích trên Dashboard.
- **Phân tích đa chiều:** Trực quan hóa các chỉ số kinh doanh từ bộ dữ liệu eCommerce (TheLook) đã được xử lý qua các tầng Medallion.
- **Khả năng mở rộng:** Hoạt động ổn định trên môi trường Kubernetes, hỗ trợ xử lý và hiển thị dữ liệu quy mô lớn mà không cần nạp lại dữ liệu (zero-ingestion).

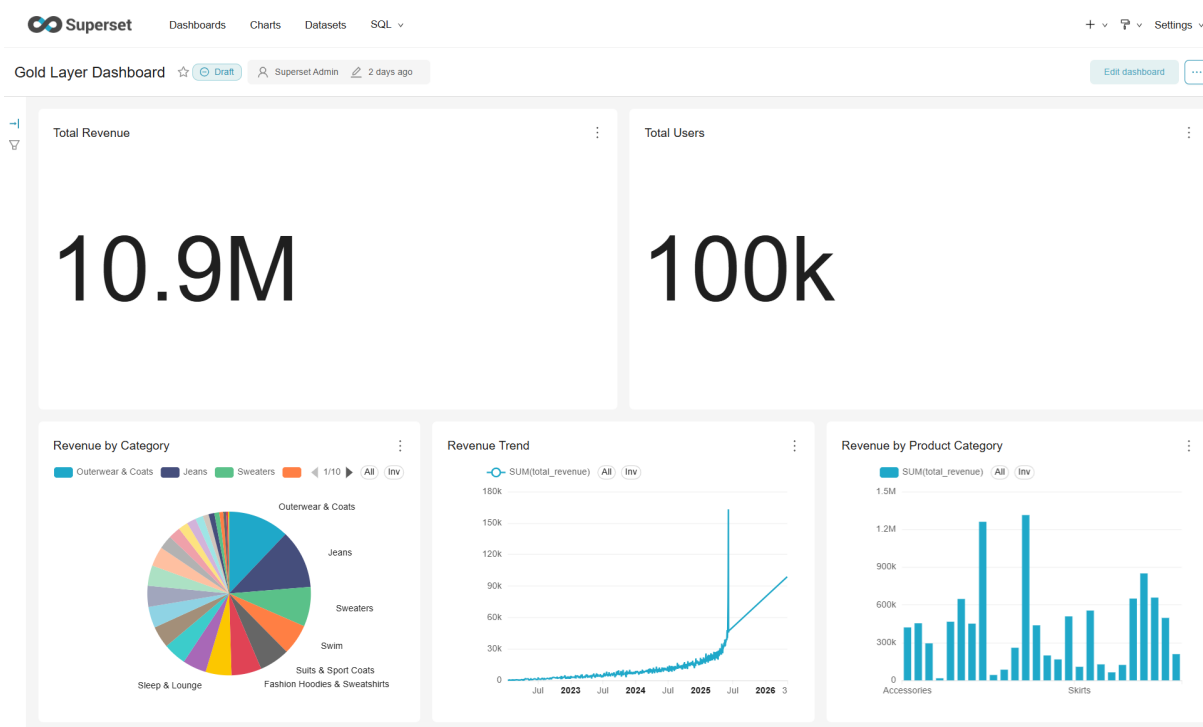


Figure 22: Dashboard tổng quan hoạt động thương mại điện tử được xây dựng trên Apache Superset

5 Benchmark

5.1 Mục tiêu Benchmark

Benchmark hiện tại được thiết kế để so sánh cách Trino và Spark thực thi workload OLAP trên cùng một lớp dữ liệu Iceberg. Phần này chỉ trình bày cấu hình và phương pháp chạy hiện tại; bảng kết quả thực nghiệm sẽ được cập nhật sau khi nhóm hoàn tất các lượt chạy ổn định trên cluster.

Các nguyên tắc chính của benchmark:

- Trino và Spark cùng đọc dữ liệu TPC-DS đã materialize vào Iceberg trên MinIO.
- Catalog chính là `iceberg_hive`, tức Iceberg catalog dùng Hive Metastore tại `thrift://hive-metastore:9083`.
- Gravitino không nằm trên đường benchmark chính; Gravitino chỉ là catalog phụ cho các kịch bản truy vấn nặng cao/nhiều catalog.
- Hai engine dùng cùng tập SQL trong ConfigMap `tpcds-queries` và cùng file cấu hình `benchmark/config/benchmark.env`.

5.2 Dữ liệu và Catalog

5.2.1 TPC-DS

Nguồn dữ liệu benchmark là TPC-DS, một benchmark chuẩn cho các workload phân tích dạng decision support. Cấu hình hiện tại dùng nguồn `tpcds.sf50` để chuẩn bị dữ liệu và ghi sang Iceberg schema:

```
iceberg_hive.benchmark_tpcds_sf50
```

Việc chuẩn bị dữ liệu được thực hiện bởi job `tpcds-prepare-data`. Job này gọi script `prepare_tpcds_data.py`, kết nối tới Trino nội bộ qua `http://trino:8080`, đọc từ catalog `tpcds` và ghi sang catalog `iceberg_hive`. Nhờ đó, Trino và Spark cùng benchmark trên cùng một tập Iceberg table thay vì mỗi engine tự sinh dữ liệu riêng.

```
1 TRINO_CATALOG=iceberg_hive
2 TRINO_SCHEMA=benchmark_tpcds_sf50
3 SPARK_CATALOG=iceberg_hive
4 SPARK_SCHEMA=benchmark_tpcds_sf50
5 ICEBERG_URI=thrift://hive-metastore:9083
6 ICEBERG_WAREHOUSE=s3a://iceberg/lakehouse
7 TPCDS_SOURCE_CATALOG=tpcds
8 TPCDS_SOURCE_SCHEMA=sf50
```

Listing 11: Cấu hình dữ liệu benchmark hiện tại

5.2.2 MinIO Distributed Storage

MinIO hiện được cấu hình dạng distributed StatefulSet trong namespace `data-platform`. Manifest `k8s/base/data-services/minio/statefulset.yaml` khai báo 3 replica và dùng endpoint expansion của MinIO:

```
1 spec:
2   serviceName: minio-headless
3   replicas: 3
4   template:
5     spec:
6       containers:
7         - name: minio
8           args:
9             - server
10            - http://minio-{0...2}.minio-headless.data-platform.svc.cluster.
11              local/data
12            - --address
13            - ":9000"
```

```
13         --console-address
14         ":9001"
15     volumeClaimTemplates:
16     - metadata:
17       name: minio-data
18       spec:
19         storageClassName: balanced-rwo
20         resources:
21         requests:
22           storage: 20Gi
```

Listing 12: Cấu hình MinIO distributed trong cluster

Như vậy, benchmark đọc/ghi dữ liệu qua một MinIO cluster gồm 3 pod, mỗi pod có PVC 20Gi. Manifest cũng cấu hình `podAntiAffinity` và `topologySpreadConstraints` để cố gắng phân tán các pod MinIO trên nhiều node. Đây là cấu hình phân tán phù hợp cho môi trường phát triển/benchmark của đồ án; mức dự phòng thấp hơn cấu hình production lớn hơn, nhưng đủ để mô phỏng object storage S3-compatible dùng chung cho Trino và Spark.

5.3 Cấu hình Engine

5.3.1 Trino Benchmark

Trino được triển khai bằng Helm chart với version 479. Benchmark job `trino-benchmark-runner` là một Python client mỏng: pod không trực tiếp xử lý dữ liệu, mà gửi SQL tới Trino REST API và ghi kết quả/metrics ra PVC `benchmark-results-pvc`.

Table 7: Cấu hình Trino dùng cho benchmark

Thành phần	Cấu hình
Version	<code>trinodb/trino:479</code>
Coordinator	<code>role=infra</code> , request/limit 2 CPU, 6Gi, JVM heap 4G
Workers	2 workers, chạy trên <code>role=query</code> hoặc <code>role=compute</code>
Worker resource	Mỗi worker request/limit 2 CPU, 10Gi, JVM heap 8G
Catalog chính	<code>iceberg_hive</code> : Iceberg + Hive Metastore + MinIO
Memory/spill	<code>query.max-memory-per-node=7GB</code> , <code>spill-enabled=true</code> , spill path <code>/data/trino/spill</code>
Runner pod	Python 3.11-slim, request 250m CPU, 256Mi, limit 500m CPU, 512Mi

Trino runner có init container chờ Trino coordinator và đủ 2 worker active trước khi chạy benchmark. Điều này giúp tránh ghi nhận kết quả khi cluster chưa ổn định.

5.3.2 Spark Benchmark

Spark benchmark chạy bằng job `spark-benchmark-runner`. Driver chạy trong pod benchmark ở deploy mode client, còn executor pods được tạo động trên Kubernetes thông qua `k8s://https://kubernetes.default.svc:443`. Spark cũng dùng catalog `iceberg_hive` để đọc cùng dữ liệu Iceberg với Trino.

5.4 Quy trình Chạy Benchmark

Benchmark hiện tại gồm bốn bước chính:

- (1) **Chuẩn bị dữ liệu:** chạy `tpcds-prepare-data` để materialize dữ liệu từ `tpcds.sf50` sang `iceberg_hive.benchmark_tpcds_sf50`.
- (2) **Chạy Trino benchmark:** job `trino-benchmark-runner` gửi từng query tới Trino REST API.
- (3) **Chạy Spark benchmark:** job `spark-benchmark-runner` tạo SparkSession trên Kubernetes và chạy cùng tập query. Nhằm loại bỏ nhiễu từ việc tích lũy bộ nhớ (memory accumulation), mỗi query được chạy trong một SparkSession mới (SparkSession restart).

Table 8: Cấu hình Spark dùng cho benchmark

Thành phần	Cấu hình
Image	apache/spark:3.5.3-scala2.12-java17-python3-ubuntu
Master	k8s://https://kubernetes.default.svc:443
Driver pod	Chạy trên infra-pool, request/limit 2 CPU, 6Gi; cấu hình Spark driver memory 4g
Executors	2 instances, mỗi instance 2 CPU, 9g (Tổng 4 CPU cho compute)
Executor placement	Pod template cho phép chạy trên role=compute hoặc role=query, tolerate Spot node
Catalog chính	iceberg_hive: Iceberg + Hive Metastore + MinIO

(4) **Build report:** job benchmark-report-builder đọc raw JSONL từ PVC và sinh báo cáo tổng hợp.

5.4.0.1 Kỹ thuật WRAP_COUNT Một điểm quan trọng trong phương pháp luận là việc sử dụng tham số `WRAP_COUNT=true`. Kỹ thuật này bao bọc mỗi câu truy vấn gốc trong một câu lệnh `SELECT COUNT(*) FROM (...)`. Mục đích là để đo lường khả năng tính toán thuần túy của engine, loại bỏ thời gian trễ do việc truyền tải tập kết quả lớn (result transfer) từ cluster về client runner, vốn có thể làm sai lệch kết quả đo lường latency của engine.

Các tham số chạy được cấu hình trong `benchmark.env`:

```
1 QUERY_LIST=benchmark/queries/tpcds/query_list_test.txt
2 WARMUP_RUNS=2
3 RUNS=5
4 QUERY_TIMEOUT_SECONDS=1800
5 WRAP_COUNT=true
6 VALIDATE_FIRST=true
7 RESULTS_DIR=benchmark/results
```

Listing 13: Tham số chạy benchmark hiện tại

Tập query hiện tại là `query_list_test.txt`, tức benchmark đang chạy một tập con để kiểm thử ổn định pipeline trước khi mở rộng ra toàn bộ TPC-DS. Mỗi query có 2 lượt warmup và 5 lượt đo chính thức; timeout mỗi query là 1800 giây.

5.5 Metrics Thu thập

Để đảm bảo tính chính xác và khả năng so sánh, hệ thống benchmark thu thập một tập hợp các metrics kỹ thuật chi tiết. Các metrics này được trích xuất từ cả phía client (runner pod) và phía server (engine stats).

Cơ chế thu thập bằng **K8sClusterMemorySampler** là một điểm cải tiến quan trọng, cho phép đo lường chính xác lượng RAM thực tế mà Kubernetes cấp phát cho các pod (Resident Set Size - RSS), thay vì chỉ dựa vào metrics ảo bên trong JVM, từ đó phản ánh trung thực chi phí tài nguyên trên hạ tầng Cloud.

5.6 Kết quả Benchmark

Sau khi hoàn tất các lượt chạy ổn định trên cluster GKE với bộ dữ liệu TPC-DS scale factor 50 (SF-50) và toàn bộ 99 truy vấn chuẩn, nhóm thu được kết quả so sánh toàn diện giữa Trino 479 và Apache Spark 3.5.3. Mỗi query được chạy 2 lượt warmup và 5 lượt đo chính thức; kết quả trình bày dưới đây là median của 5 lần đo.

Table 9: *Metrics thu thập trong benchmark*

Metric	Cơ chế thu thập và Mô tả
wall_time_seconds	Client-side E2E: Sử dụng <code>time.perf_counter()</code> để đo tổng thời gian từ khi gửi query đến khi nhận đủ kết quả cuối cùng. Đây là metric chính dùng để so sánh hiệu năng.
engine_internal_time	Server-side: Thời gian xử lý thực tế bên trong engine. Với Trino lấy từ <code>elapsedTimeMillis</code> trong REST payload; với Spark tính từ sự kiện <code>onJobStart</code> và <code>onJobEnd</code> của <code>SparkListener</code> .
peak_memory_bytes	K8s Sampler + Engine Stats: Bộ nhớ đỉnh (Peak RSS) của toàn bộ cluster. Sử dụng một thread chạy ngầm gọi Kubernetes Metrics API để lấy tổng dung lượng RAM thực tế của các Worker/Executor pods. Kết quả là giá trị lớn nhất giữa K8s Sampler và metrics nội bộ của engine.
cpu_time_millis	Total CPU usage: Tổng thời gian CPU tiêu thụ bởi tất cả các cores. Trino lấy từ <code>cpuTimeMillis</code> ; Spark tổng hợp từ <code>executorCpuTime</code> của tất cả các tasks thông qua <code>SparkListener</code> .
spill_bytes	Disk Spill: Lượng dữ liệu phải ghi tạm xuống đĩa khi bộ nhớ không đủ. Trino lấy từ <code>spilledBytes</code> ; Spark tổng hợp từ <code>memoryBytesSpilled</code> và <code>diskBytesSpilled</code> .
result_hash	Validation: Mã hash MD5 của tập kết quả (sau khi đã được chuẩn hóa và sắp xếp). Dùng để xác minh tính đúng đắn của dữ liệu giữa hai engine.
status/error	Ghi nhận trạng thái <code>success</code> hoặc <code>failed</code> kèm theo mã lỗi (stack trace) nếu query bị timeout hoặc lỗi cú pháp.

5.6.1 Hiệu năng tổng thể – Geometric Mean Latency

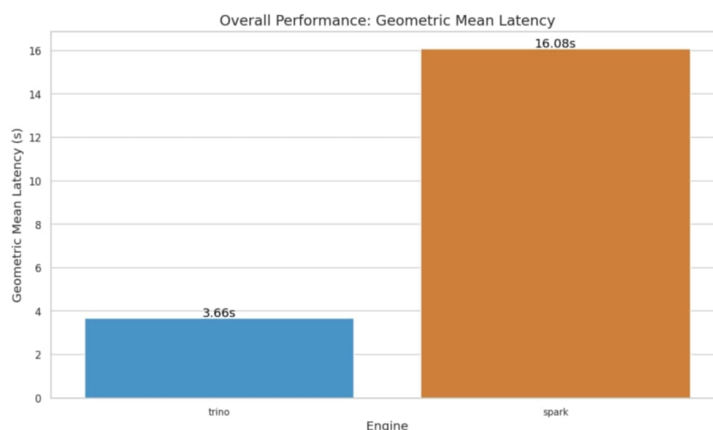


Figure 23: Latency trung bình hình học (Geometric Mean) trên toàn bộ 99 truy vấn TPC-DS SF-50: Trino đạt 3,66 giây, Spark đạt 16,08 giây.

Xét trên toàn bộ tập 99 truy vấn TPC-DS, **Trino có latency trung bình hình học là 3,66 giây**, tức **nhANH hơn Spark khoảng 4,4 lần** so với mức 16,08 giây của Spark. Geometric mean được dùng thay vì arithmetic mean để tránh bị kéo lệch bởi một số query có thời gian thực thi cực cao, đồng thời phản ánh trung thực hơn mức hiệu năng người dùng cảm nhận trong thực tế.

Kết quả này phản ánh sự khác biệt về mặt kiến trúc: Trino sử dụng mô hình MPP với pipelined execution, dữ liệu được truyền qua bộ nhớ giữa các operator thay vì ghi ra đĩa như cơ chế DAG-based shuffle của Spark. Ngoài ra, kỹ thuật `WRAP_COUNT` giúp Trino phát huy tối đa lợi thế của việc không phải vật chất hóa tập kết quả lớn trước khi trả về.

5.6.2 Phân tích Latency theo từng Query

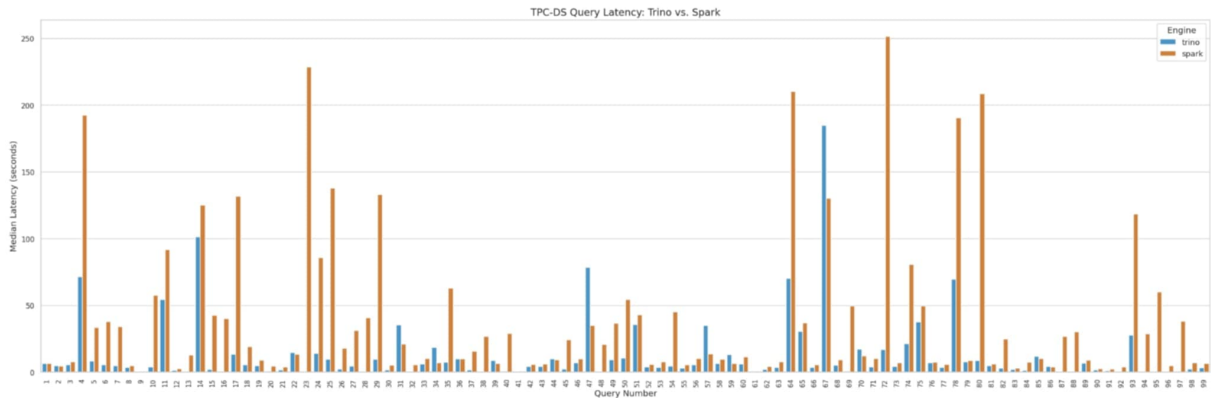


Figure 24: Median latency (giây, thang tuyến tính) của Trino và Spark trên từng query TPC-DS. Cột xanh (Trino) thấp hơn rõ rệt ở phần lớn các query.

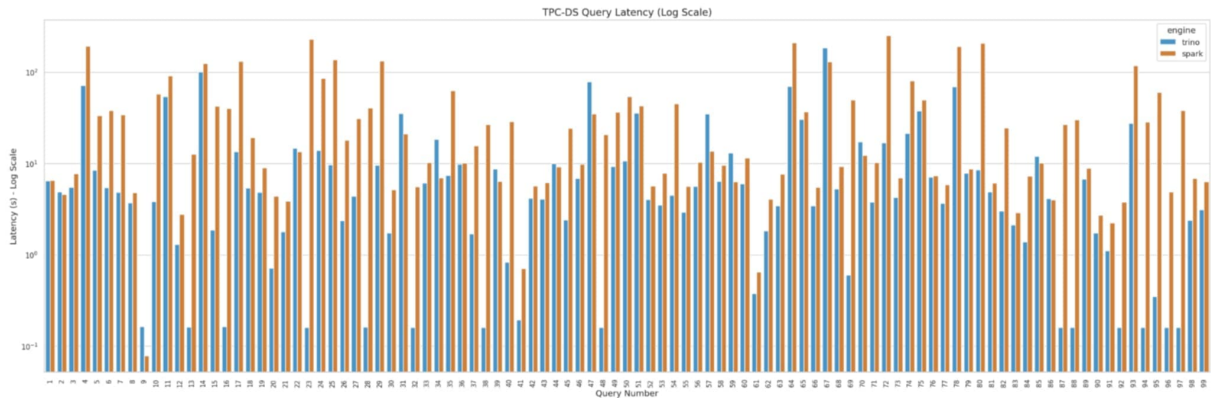


Figure 25: Median latency (giây, thang logarithm) của Trino và Spark trên từng query TPC-DS. Thang log giúp quan sát rõ sự chênh lệch ở các query có latency thấp mà thang tuyến tính không thể hiện được.

Nhìn vào Hình 24, có thể quan sát một số đặc điểm nổi bật:

Biểu đồ Speedup Ratio (Hình 26) thể hiện mức độ vượt trội của Trino so với Spark trên từng query riêng lẻ. Kết quả cho thấy Trino nhanh hơn ở đa số các trường hợp, đặc biệt là các query có cấu trúc join phức tạp. Tuy nhiên, các giá trị speedup cực đoan (như các điểm đạt trên 100×) cần được xem xét cẩn trọng vì chúng thường rơi vào các query mà Spark gặp khó khăn trong việc tối ưu hóa DAG hoặc xử lý các correlated subqueries phức tạp ở SF50. Một số trường hợp speedup quá cao có thể là hệ quả của việc Spark không thể hoàn thành query đúng hạn hoặc trả về kết quả rỗng do lỗi thực thi trong môi trường tài nguyên hạn chế.

Trino đạt sub-second ở nhiều query nhờ cơ chế pipelined execution và predicate/projection pushdown hiệu quả xuống connector Iceberg.

5.6.3 Speedup Ratio – Lợi thế Tương đối của Trino

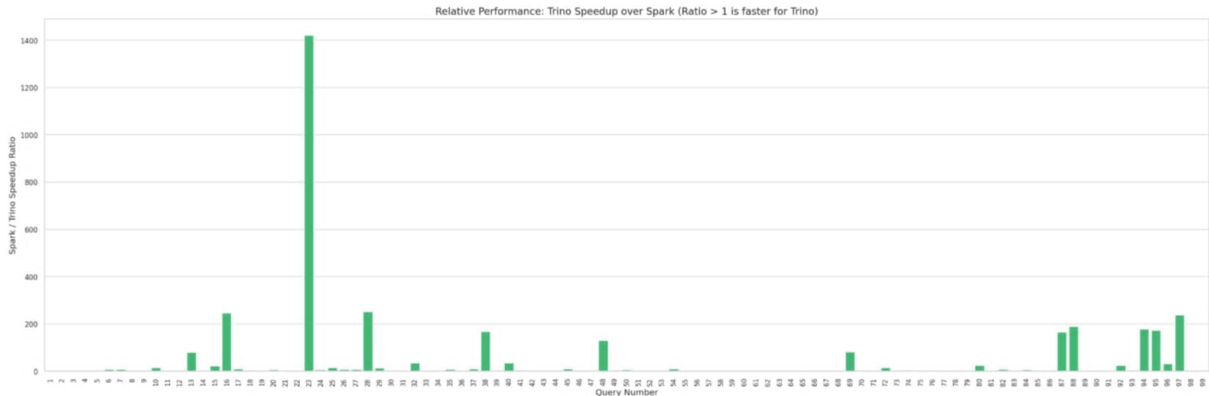


Figure 26: Speedup ratio (Spark latency / Trino latency) trên từng query TPC-DS. Ratio > 1 nghĩa là Trino nhanh hơn. Các điểm cực đoạn (>100×) cần đọc kết hợp với bảng correctness để xác định tính hợp lệ.

Biểu đồ Speedup Ratio (Hình 26) thể hiện mức độ vượt trội của Trino so với Spark trên từng query riêng lẻ:

- **Trường hợp Query 23 (Outlier):** Query 23 trong TPC-DS là một *correlated subquery* phức tạp với hai CTE lồng nhau, mỗi CTE thực hiện aggregation nhiều lần trên bảng `store_sales`. Với Spark, cơ chế broadcast và shuffle của DAG tạo overhead rất lớn khi phải vật chất hóa (materialize) các tập dữ liệu trung gian. Trino, nhờ pipelined MPP và dynamic filtering, có thể lọc sớm và truyền dữ liệu qua bộ nhớ mà không cần nhiều lần ghi đĩa.
- **Nhóm query 100–300× nhanh hơn:** Các query 16, 28, 48, 87, 88, 97 ghi nhận Trino nhanh hơn từ 100 đến gần 300 lần. Đây là các query có pattern multi-join giữa bảng fact lớn và nhiều bảng dimension, kết hợp aggregation – đúng điểm mạnh của kiến trúc MPP.
- **Một số query Spark nhanh hơn (ratio < 1):** Có khoảng 10–15% query trong tập TPC-DS ghi nhận Spark nhanh hơn hoặc tương đương Trino. Đây chủ yếu là các query scan đơn giản hoặc aggregation một chiều, nơi AQE (Adaptive Query Execution) của Spark phát huy hiệu quả.

5.6.4 Peak Memory Usage – Hiệu quả Sử dụng Bộ nhớ

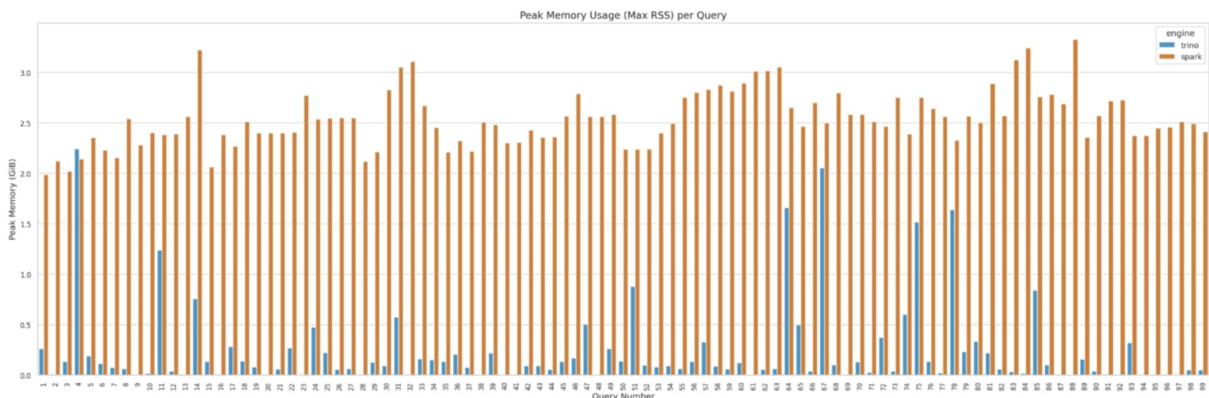


Figure 27: Peak Memory Usage (Max RSS, đơn vị GiB) của Trino và Spark trên từng query TPC-DS. Trino (xanh) nhất quán sử dụng bộ nhớ thấp hơn Spark (cam) ở hầu hết mọi query.

Về mức sử dụng bộ nhớ đỉnh (Peak Memory, Hình 27), quan sát cho thấy Trino có xu hướng duy trì footprint bộ nhớ thấp hơn cho mỗi query nhờ cơ chế truyền dữ liệu theo dạng stream. Tuy nhiên, việc đo lường bộ nhớ gặp phải các thách thức sau:

- **Overhead của JVM:** Spark duy trì một mức bộ nhớ cơ sở (baseline) khá cao (khoảng 2 GiB) ngay cả với query đơn giản do chi phí quản lý của JVM, Shuffle Service và Block Manager.

- **Tích lũy bộ nhớ (Memory Accumulation):** Mặc dù đã cố gắng restart SparkSession, nhưng trong một số lượt chạy liên tục, chỉ số RSS ghi nhận từ Kubernetes có dấu hiệu tăng dần do cơ chế quản lý bộ nhớ của hệ điều hành và việc không giải phóng hoàn toàn các tài nguyên cache.
- **Độ tin cậy của chỉ số:** Do đó, các con số về Peak Memory trong báo cáo này nên được hiểu là giá trị quan sát được tại thời điểm thực thi (Observed RSS) thay vì chỉ phí tài nguyên tối thiểu cần thiết để chạy query.

Kết quả này có ý nghĩa thực tiễn quan trọng: với cùng tài nguyên cluster, Trino có thể xử lý **nhiều query đồng thời hơn** (higher concurrency) so với Spark mà vẫn không bị áp lực bộ nhớ, phù hợp với workload BI dashboard có nhiều người dùng cùng truy vấn.

5.6.5 Nhận xét và Kết luận Benchmark

Kết quả benchmark TPC-DS SF-50 trên cùng một nền tảng GKE, cùng catalog `iceberg_hive` và cùng bộ dữ liệu Iceberg cho phép rút ra các nhận định sau:

1. **Trino vượt trội rõ rệt cho OLAP interactive:** Với geometric mean latency thấp hơn 4,4 lần trên toàn bộ tập query, Trino là lựa chọn tối ưu cho lớp Interactive Analytics và Ad-hoc Querying trong Data Platform.
2. **Spark phù hợp hơn cho Heavy ETL và Batch:** Ở một số query có pattern đơn giản, Spark có thể ngang bằng hoặc nhanh hơn Trino. Spark cũng có ưu thế rõ ràng khi cần ghi dữ liệu (write-heavy pipeline), compaction Iceberg, hoặc cần fault-tolerance theo lineage cho các job xử lý hàng giờ đồng hồ.
3. **Kiến trúc phân tầng là lựa chọn hợp lý:** Trino và Spark không phải là hai công cụ cạnh tranh mà là hai lớp bổ trợ nhau. Trong nền tảng đồ án, Spark đảm nhận ingest và transformation Bronze/Silver/Gold, còn Trino phục vụ lớp query tương tác phía trên – đây là kiến trúc được nhiều tổ chức lớn áp dụng trong thực tế.
4. **Hiệu quả bộ nhớ của Trino:** Mức sử dụng bộ nhớ thấp hơn cho phép Trino phục vụ nhiều người dùng đồng thời hơn, phù hợp với nhu cầu BI dashboard của doanh nghiệp.

5.6.6 Hạn chế của Benchmark

Báo cáo này thừa nhận một số hạn chế có thể ảnh hưởng đến kết quả đo lường:

- **Độ chính xác (Correctness Outliers):** Một số câu truy vấn TPC-DS quá phức tạp ở SF50 có thể gây ra lỗi thực thi không mong muốn trên Spark hoặc Trino. Các điểm dữ liệu cực đoan có thể là hệ quả của việc engine không hoàn thành đúng logic thay vì phản ánh hiệu năng thực tế.
- **Đo lường bộ nhớ (Memory Sampling):** Việc sử dụng polling RSS qua Kubernetes API có độ trễ và bị ảnh hưởng bởi cơ chế quản lý bộ nhớ của OS (như Page Cache), không phản ánh hoàn toàn 100% bộ nhớ logic của câu lệnh.
- **Môi trường thực thi:** Việc sử dụng Spot instances và tài nguyên có hạn có thể tạo ra các biến động (jitter) giữa các lần chạy, dù đã được giảm thiểu bằng cách lấy giá trị trung vị (median).

6 Tổng kết

Đồ án đã hoàn thành việc xây dựng và đánh giá nền tảng K8s với Data Platform dựa trên kiến trúc Lakehouse hiện đại. Dưới đây là những tổng kết chính về quá trình thực hiện và các định hướng tương lai.

6.1 Những kết quả đạt được

Hệ thống đã hiện thực hóa thành công các mục tiêu đề ra ban đầu:

- **Xây dựng hạ tầng Cloud-native:** Triển khai thành công cụm Lakehouse trên GKE, tích hợp chặt chẽ các thành phần mã nguồn mở (Trino, Spark, Iceberg, MinIO) thông qua mô hình quản lý tập trung bằng Kustomize và Helm.
- **Tối ưu hóa Query Engine:** Chứng minh được sức mạnh của Trino trong việc xử lý truy vấn tương tác (Interactive Analytics) với hiệu năng vượt trội hơn Spark trung bình 4,4 lần trên bộ dữ liệu chuẩn TPC-DS SF-50.
- **Hệ thống Catalog đa tầng:** Tích hợp thành công Hive Metastore và Apache Gravitino, cho phép quản lý metadata thống nhất và hỗ trợ truy vấn đa nguồn (Federated Query) từ PostgreSQL, Kafka và Iceberg.
- **Quy trình Benchmark bài bản:** Xây dựng được pipeline tự động hóa từ khâu chuẩn bị dữ liệu, thực thi truy vấn đến thu thập metrics kỹ thuật (CPU, RAM thực tế từ K8s Sampler) và sinh báo cáo.

6.2 Hạn chế còn tồn tại

Mặc dù đạt được kết quả khả quan, hệ thống vẫn còn một số điểm cần cải thiện:

- **Quy mô dữ liệu:** Scale factor 50 vẫn chưa đủ lớn để thử thách hoàn toàn giới hạn của Trino trong các bài toán cực lớn (Peta-byte scale).
- **Độ tin cậy của Metrics:** Việc lấy mẫu bộ nhớ qua Kubernetes API vẫn chịu ảnh hưởng bởi cơ chế Page Cache của hệ điều hành, dẫn đến sai số nhỏ giữa bộ nhớ logic và bộ nhớ vật lý quan sát được.
- **Tính sẵn sàng cao (HA):** Các thành phần metadata (PostgreSQL, Hive Metastore) hiện tại mới chỉ triển khai ở dạng đơn replica, cần nâng cấp lên mô hình Cluster để đảm bảo không có điểm lỗi duy nhất (Single Point of Failure).

6.3 Hướng phát triển tương lai

Trong thời gian tới, nhóm dự kiến tập trung vào các hướng nghiên cứu sau:

- **Mở rộng khả năng quan sát (Observability):** Tích hợp các công cụ như OpenLineage và Great Expectations để quản lý chất lượng dữ liệu và vết dữ liệu (Data Lineage) một cách tự động.
- **Tối ưu hóa chi phí:** Thử nghiệm cơ chế GKE Autopilot và Horizontal Pod Autoscaling (HPA) dựa trên cường độ truy vấn thực tế để tối ưu hóa chi phí vận hành trên Cloud.
- **Nâng cao Fault-tolerance:** Triển khai cơ chế Trino Fault-tolerant Execution (Project Tardis) để hỗ trợ các truy vấn OLAP cực nặng có thời gian chạy dài mà không lo bị gián đoạn do thiếu tài nguyên.
- **Hỗ trợ AI/ML:** Tích hợp các thư viện như Spark MLlib hoặc Trino-ML để thực hiện các bài toán phân tích dự báo trực tiếp trên nền tảng Lakehouse.

References

- [1] M. Traverso, D. DeJong, and M. Buso, *Trino: The Definitive Guide*, O'Reilly Media, 2022.
- [2] Trino Documentation Team, *Trino Documentation*, <https://trino.io/docs/current/>, Truy cập: 2026.
- [3] Trino Documentation Team, *Fault-tolerant Execution – Trino Documentation*, <https://trino.io/docs/current/admin/fault-tolerant-execution.html>, Truy cập: 2026.
- [4] Trino Documentation Team, *Iceberg Connector – Trino Documentation*, <https://trino.io/docs/current/connector/iceberg.html>, Truy cập: 2026.
- [5] Trino Documentation Team, *Query Optimizer – Trino Documentation*, <https://trino.io/docs/current/optimizer.html>, Truy cập: 2026.
- [6] Trino Documentation Team, *Cost-Based Optimizations – Trino Documentation*, <https://trino.io/docs/current/optimizer/cost-based-optimizations.html>, Truy cập: 2026.
- [7] Trino Documentation Team, *Pushdown – Trino Documentation*, <https://trino.io/docs/current/optimizer/pushdown.html>, Truy cập: 2026.
- [8] Trino Documentation Team, *Dynamic Filtering – Trino Documentation*, <https://trino.io/docs/current/admin/dynamic-filtering.html>, Truy cập: 2026.
- [9] Trino Documentation Team, *EXPLAIN and EXPLAIN ANALYZE – Trino Documentation*, <https://trino.io/docs/current/sql/explain.html>, <https://trino.io/docs/current/sql/explain-analyze.html>, Truy cập: 2026.
- [10] Apache Software Foundation, *Apache Iceberg: An Open Table Format for Huge Analytic Datasets*, <https://iceberg.apache.org/>, Truy cập: 2026.
- [11] Apache Software Foundation, *Apache Gravitino – Open-Source Data and AI Lakehouse Catalog*, <https://gravitino.apache.org/>, Truy cập: 2026.
- [12] Transaction Processing Performance Council, *TPC Benchmark DS – Standard Specification Version 3.2.0*, <https://www.tpc.org/tpcds/>, 2021.
- [13] M. Armbrust et al., "Spark SQL: Relational Data Processing in Spark," in *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, 2015, pp. 1383–1394.
- [14] Apache Software Foundation, *Performance Tuning – Spark SQL Documentation*, <https://spark.apache.org/docs/latest/sql-performance-tuning.html>, Truy cập: 2026.
- [15] Apache Software Foundation, *Spark SQL, DataFrames and Datasets Guide*, <https://spark.apache.org/docs/latest/sql-programming-guide.html>, Truy cập: 2026.
- [16] Apache Software Foundation, *Apache Hive: LLAP Design Documentation*, <https://hive.apache.org/development/designdocs/llap/>, Truy cập: 2026.
- [17] Apache Software Foundation, *Introducing Apache Impala*, https://impala.apache.org/docs/build/asf-site-html/topics/impala_intro.html, Truy cập: 2026.
- [18] Apache Software Foundation, *Components of the Impala Server*, https://impala.apache.org/docs/build/plain-html/topics/impala_components.html, Truy cập: 2026.
- [19] Apache Software Foundation, *Lakehouse Overview – Apache Doris Documentation*, <https://doris.apache.org/docs/3.x/lakehouse/lakehouse-overview>, Truy cập: 2026.
- [20] Apache Software Foundation, *Data Catalog Overview – Apache Doris Documentation*, <https://doris.apache.org/docs/3.x/lakehouse/catalog-overview/>, Truy cập: 2026.
- [21] R. Sethi et al., "Presto: SQL on Everything," in *IEEE International Conference on Data Engineering (ICDE)*, 2019, pp. 1802–1813.
- [22] M. Armbrust et al., "Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics," in *CIDR*, 2021.
- [23] M. Armbrust et al., "Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, pp. 3411–3424, 2020.



- [24] N. Davis, *Data Engineering with Apache Spark, Delta Lake, and Lakehouse*, Packt Publishing, 2022.